

图 5-24

(a) 一个拥塞的网络；(b) 网络中不拥塞的部分（图中显示了从 A 到 B 的一条虚电路）

5.3.4 流量调节

在 Internet 和许多其他计算机网络中，发送方调整它们的传输速度以便发送网络能实际传送的流量。在这种设置下，网络的目标是在拥塞发生之前正常工作。而当拥塞迫在眉睫，它必须告诉发送方紧急刹车放慢传输速度。这种反馈是一种常态而不是针对特殊情况的一种处理。术语**拥塞避免**（congestion avoidance）有时用来对比某个操作点，以示与已经变得（过度）拥挤的网络区分开。

现在让我们考查一些限制流量的方法，这些方法可用在数据报网络和虚电路网络。每个方法必须解决两个问题。首先，路由器必须确定何时快要接近拥塞，最好在拥塞发生之前能确定。为此，每个路由器可连续监测它正在使用的资源。三种可能的资源分别是输出线路的利用率、在路由器内缓冲的排队数据包，以及由于没有足够的缓冲而丢失的数据包数量。在这些可能性中，第二个是最有用的。平均利用率并没有直接考虑大多数流量的突发——50%的利用率对平滑流量来说或许很低，但对于变化很大的流量来说就太高了。丢失数据包的计数来得太迟。在数据包丢失时拥塞早就已经形成。

路由器内部的排队延迟直接捕获了数据包经历过的任何拥塞情况。它在大部分时间应该很低，但当有一个突发流量产生积压时会跳跃。为了维持良好的排队延迟估计 d ，假设 s 表示瞬时队列长度的样值，则 d 可定期生成，并按如下方式进行更新

$$d_{\text{new}} = \alpha d_{\text{old}} + (1-\alpha)s$$

其中常数 α 决定路由器多快忘记最近的历史。这就是所谓的**指数加权移动平均**（EWMA, Exponentially Weighted Moving Average）。它能平滑流量的波动，相当于一个低通滤波器。每当 d 升高到某个阈值之上，路由器就要注意开始拥塞了。

要考虑的第二个问题是，路由器必须及时把反馈信息传递给造成拥塞的发送方。拥塞是网络的一种体验，但缓解拥塞则需要使用网络的发送方采取行动。为了传递反馈信息，路由器必须标识适当的发送方；然后提醒它们小心谨慎，别向本已拥挤的网络发送更多的数据包。不同的方案使用不同的反馈机制，我们马上对此分别描述。

抑制包

通知拥塞发送方的最直接方式是直接告诉发送方。在这种方法中，路由器选择一个被拥塞的数据包，给该数据包的源主机返回一个抑制包（choke packet）。抑制包中的目标地址取自该拥塞数据包。同时，在原来的拥塞数据包上添加一个标记（设置头部中的一位），

因而它在前行的路径上不会产生更多的抑制包。除此以外，数据包的转发过程如同平常一样。

当源主机收到了抑制包，按照要求它必须减少发送给指定目标的流量，比如说减少50%。在数据报网络中，发生拥塞时路由器简单地随机选择一个数据包，很有可能就把抑制包发给了快速发送方，因为发送方的速度越快，它的数据包排队在路由器队列中的数目就越多。这个协议隐含的反馈有助于防止拥塞，但又不会抑制任何发送方，除非真的发生了拥塞。出于同样的原因，很可能多个抑制包被发送到了一个给定的主机和目的地。主机应该忽略掉在固定时间间隔内到达的这些额外抑制包，直至其减缓流量的行为产生了效果。超过这个固定的时间间隔，从路由器进一步反馈来的抑制包则指出网络仍然处于拥塞状态。

早期 Internet 使用的一个抑制包例子是 SOURCE-SEQUENCE 消息 (Postel, 1981)。尽管它从来没有流行起来，不过，部分原因在于它产生的情况和效果都没有明确说明。现代 Internet 使用了另外一种“通知”设计方案，我们下面就描述它。

显式拥塞通知

除了生成额外的包发出拥塞警告外，路由器可以在它转发的任何数据包上打上标记（设置数据包头的某一个标志位）发出信号，表明它正在经历着拥塞。当网络传递数据包时，接收方可以注意到有个拥塞已经发生，在它发送应答包时顺便告知发送方。然后发送方可以像以前那样紧急刹车降低传输速率。

这种设计方案称为显式拥塞通知 (ECN, Explicit Congestion Notification)，且已被用在 Internet 上 (Ramakrishnan 等, 2001)。这是早期拥塞信令协议的细化，尤其是对二进制反馈方案 (Ramakrishnan 和 Jain, 1988) 的细化，该方案曾经用在 DECNET 体系结构。IP 数据包头中的两位用来记录数据包是否经历了拥塞。数据包从源端发出时没有标记，如图 5-25 所示。如果它们通过的任何一个路由器正经历着拥挤，该路由器在转发数据包时将其标记为经历拥塞；然后接收方在它的下一个应答数据包里回显该标志作为显式拥塞信号。这显示为图中的一条虚线，它表明在 IP 层以上（如在 TCP）。发送方必须紧踩油门控制传输，如同采用抑制包的情况。

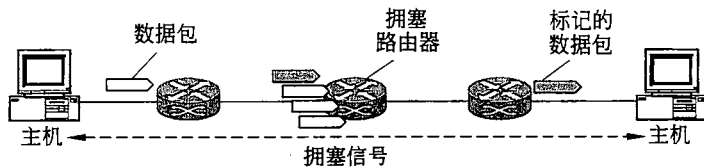


图 5-25 显式拥塞通知

逐跳后压

当网络速度很高或者距离很远时，由于传播延迟的缘故，拥塞信号发出后到它产生作用这期间又有许多新的数据包已经被注入到网络。例如，请考虑这样的情形：旧金山的一台主机（图 5-26 中的路由器 A）正在给纽约的一台主机（图 5-26 中的路由器 D）发送数据，速度为 OC-3 的 155 Mbps。如果纽约的主机现在用完了缓冲空间，它将花 40 毫秒的时间才能将抑制包发回给旧金山主机，告诉它降低传输速度。如果采用 ECN 指示，则需要更

长的时间，因为抑制消息通过接收方传递。抑制包的传播过程如图 5-26 (a) 中的第 2、3、4 步所示。在这 40 毫秒期间，A 又给 D 发送了 6.2 Mb 的数据。即使旧金山的主机立刻停机，传输管道里的 6.2 Mb 数据也将连续倾入网络，而且网络必须要对它们进行处理。只有在图 5-26 (a) 的第 7 个图中，纽约的路由器才会注意到数据包流减慢了。

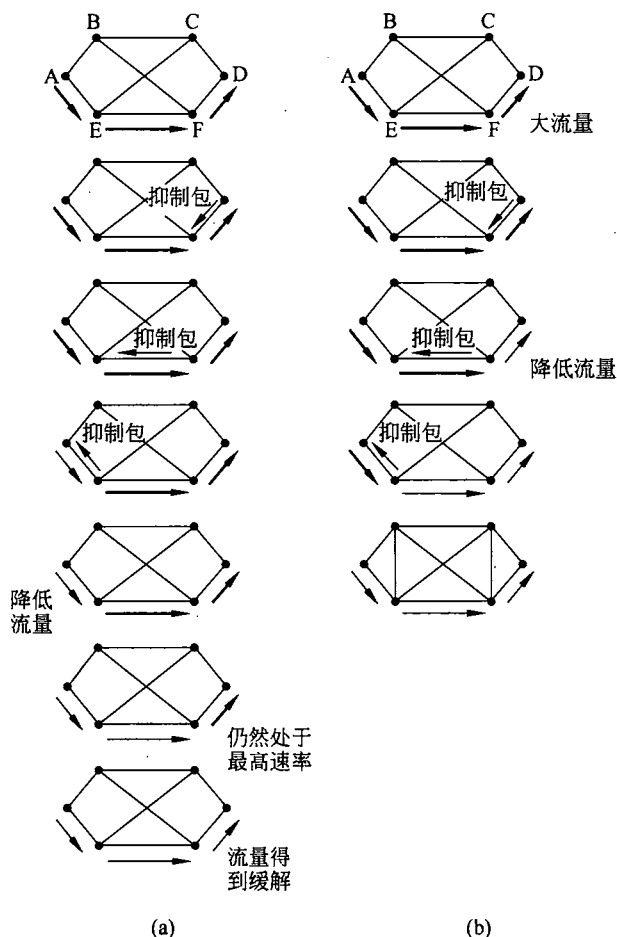


图 5-26

(a) 抑制包只影响源端；(b) 抑制包影响途径的每一跳

另一种办法是让抑制包在沿途的每一跳都发挥作用，如图 5-26 (b) 中的序列所示。在这里，只要抑制包到达 F，则 F 必须按照要求减慢发给 D 的数据包流。这样做的结果是要求 F 为 D 的数据包流分配更多的缓冲区，因为源主机仍然在全速发送数据，但是 F 这么做却让 D 立刻得到缓解，就好像电视广告中的头痛疗法一样。在下一步，抑制包到达 E，它告诉 E 减慢给 F 的数据包流。这一行动给 E 的缓冲区带来更大需求，但是却让 F 立即得到缓解。最后，抑制包到达 A，数据包流才真的减慢下来。

这种逐跳方案的实际效果是拥塞点上的拥塞现象很快得到了缓解，但是其代价是上游路径需要消耗更多的缓冲区空间。使用这种方法可以将拥塞消灭在萌芽状态，而不会丢失任何数据包。有关该算法思想的详细讨论请参考 (Mishra 等, 1996)。

5.3.5 负载脱落

当以上任何一种方法都无法消除拥塞时，路由器可以亮出它的杀手锏，即负载脱落。负载脱落（load shedding）是一种富有想象力的说法，它指当路由器因为来不及处理数据包而面临被这些数据包淹没的危险时，就将它们丢弃。此术语来源于电力发电领域，在炎热的夏日，当电力需求超过了供电能力，为了避免电力系统崩溃而有意切断某些特定区域的电力供给。

对于一个被数据包淹没的路由器来说，关键的问题是选择丢弃哪个数据包。首选的方案可能取决于使用网络的应用程序类型。对于文件传输，旧的数据包价值要高于新的数据包。这个原因我们可以用一个例子加以说明。如果丢弃数据包6，而保持数据包7~10，只会迫使接收方做更多的工作来缓冲它已经接收但尚不能使用的数据。相比之下，对于实时媒体流，新的数据包价值超过老的数据包。因为如果数据包被延迟并且错失了给用户的播放时间，那么该数据包就变得一无所用。

前一种策略（即旧的比新的好）通常称为葡萄酒（wine）策略，而后一种策略（即新的比旧的好）通常称为牛奶（milk）策略，因为大多数人更愿意饮用新鲜牛奶和品尝陈年葡萄酒。

更智能的卸载方式需要发送方的合作。一个例子是携带路由信息的数据包。这些数据包比普通数据包要重要得多，因为它们是被用来建立的路由；如果丢失它们，或许就会丢失网络连接。另一个例子是视频压缩算法，如MPEG，这些算法定期发送全帧，然后发送一系列与上一个全帧的差异帧。在这种情况下，应该优先丢弃那些属于差异帧的数据包，而不能丢弃属于全帧的数据包，因为未来数据包的工作依赖于它们之前的那个全帧。

为了实现智能丢弃政策，应用程序必须在它们的数据包上打上标记，指示网络它们有多重要。然后，当不得不丢弃数据包时，路由器可以首先丢弃重要性最轻一类数据包，然后是次重要一类数据包，以此类推。

当然，除非有一些明显的激励措施来避免每个数据包都标志成“非常重要”——“永远不要丢弃”，否则没有人会愿意把自己的数据包标示成不那么重要。通常计费 and 金钱可用来阻止轻浮的标志。例如，如果发送方购买了某项服务，该服务规定了一定的发送速度，网络可能允许发送方的发送速度超过他们购买的额度，只要他们把超出部分的数据包标记为低优先级。这种策略其实并不是一个坏主意，因为它可以更加有效地利用闲置资源；只要没有其他人对此有兴趣，主机就可以使用这些资源，但不能给它们在网络困难时也拥有这种权利。

随机早期检测

在拥塞刚出现苗头时就处理它比等拥塞形成之后再设法解决它更加有效。这一观察导致了一个针对负载脱落的有趣转折，即在所有的缓冲空间都精疲力竭之前丢弃数据包。

这一观点的动机在于大多数Internet主机没有从路由器获得ECN形式的拥塞信号。相反，主机从网络能获得的唯一可靠的拥塞迹象是丢包。毕竟，很难构造出一个路由器，在它超负荷工作时能做到不丢包。因此，诸如TCP这样的传输协议硬性规定了对丢包现象做

出拥塞反应，减缓源端的响应。这背后的逻辑推理是 TCP 是专为有线网络设计的，并且有线网络非常可靠，所以丢包大多是由于缓冲区溢出而不是传输错误造成的。无线链路必须恢复链路层传输错误（所以它们没有看到在网络层）以便 TCP 能正常工作。

可以利用这种情形来帮助缓解拥塞。在局面变得毫无希望之前让路由器提前丢包，但这里有个时间点的确定问题，即发送方何时采取行动以免为时过晚。解决这个问题的一种流行算法称为随机早期检测（RED, Random Early Detection）（Floyd 和 Jacobson, 1993）。为了确定何时开始丢弃数据包，路由器要维护一个运行队列长度的平均值。当某条链路上的平均队列长度超过某个阈值时，该链路就被认为即将拥塞，因此路由器随机丢弃一小部分数据包。随机选择丢弃的数据包使得快速发送方发现丢包的可能性更大；因为在数据报网络中，路由器不能分辨出哪个源引起了网络的最大麻烦，因此随机选择丢弃的数据包或许是最佳选择。当没有出现期待的确认信息时，受此影响的发送方就会发现丢包，然后传输协议将放慢速度。因此，丢失的数据包起到了传递抑制包的同样作用，但却是隐含的，无须路由器发送任何显式信号。

相比那些只在缓冲区溢出才丢包的路由器，RED 路由器能提高网络性能，虽然它们可能需要调整正常工作方式。举例来说，理想的丢包数量取决于有多少发送方必须得到拥塞通知。然而，如果 ECN 可用，那么它就是首选的选项。它的工作方式几乎完全一样，但提供了一个显式拥塞信号而不是依据丢包来判断是否拥塞；RED 用在主机不能接收显式信号的环境里。

5.4 服务质量

我们在上一节考查的技术主要用来减少拥塞并提高网络性能。然而，存在一些应用（和客户），它们对网络的性能保障有很强需求，而不仅仅只是“在当前情况下尽力而为”。特别是多媒体应用往往需要具备最小延迟和最大吞吐量条件才能正常工作。在本节中，我们将继续我们的网络性能研究，但现在更加注重如何提供与应用需求相匹配的服务质量。这是一个 Internet 正在经历长期变革改进的领域。

提供良好服务质量的一个简单解决方案，就是建设有足够容量的网络，无论扔给它什么样的流量都能承担。这种解决方案的名称是过度配置（overprovisioning）。假设有一个体面的路由方案，那么这种网络将承载应用流量而没有重大丢失，并且以低延迟传递数据包。网络性能不会比这更好。在一定程度上，电话系统就是过度配置的一个例子，因为拿起电话而没有拨号音的情况很罕见。这里有太多的可用容量，所以需求总是能够得到满足。

这种解决方法的麻烦在于成本太昂贵。它基本上是用一堆钱来解决问题。服务质量机制让一个小容量网络以较低的成本来满足应用需求的解决途径。此外，过度配置基于预期的流量模式。如果流量模式变化太大，那么所有的赌注都将一去不回。有了服务质量机制，网络可以兑现其所做的性能保证，即使在流量高峰期拒绝了一些请求。

确保服务质量必须解决如下 4 个问题：

- （1）应用程序需要网络什么样的质量？
- （2）如何规范进入网络的流量？

- (3) 为了保障性能如何在路由器预留资源？
- (4) 网络能否安全地接受更多流量？

没有一种单一的技术能有效地解决上述所有问题。相反，研究人员已经陆陆续续开发出许多可用在网络层（和传输层）的各种技术。实际的服务质量解决方案要结合多种技术。为此，我们将介绍 Internet 服务质量的两个版本，即综合服务和区分服务。

5.4.1 应用需求

从一个源端发到一个接收方的数据包流称为一个流（flow）（Clark，1988）。在面向连接的网络中，一个流或许是一个连接上的全部数据包；而在无连接网络中，一个流是从一个进程发到另一个进程的所有数据包。每个流的需求可由四个主要参数来表示：带宽、延迟、抖动和丢失。总之，这些参数决定了一个流要求的服务质量（QoS，Quality of Service）。

图 5-27 列出了几种常见的应用和它们分别对网络质量的严格要求。请注意，网络需求小于应用需求，在这些情况下，应用程序可以改善由网络提供的服务。特别是，网络并不需要为可靠的文件传输真正做到无丢失，也并不需要为音频和视频的播放传递具有相同延迟的数据包。丢失部分可以通过重传来修复，一些抖动可以通过接收端缓冲数据包来平滑。然而，如果网络提供的带宽太少或者延迟太大，则无论应用程序做什么都无法补救这种情况。

应用	带宽	延迟	抖动	丢失
电子邮件	低	低	低	中等
文件共享	高	低	低	中等
浏览网页	中等	中等	低	中等
远程登录	低	中等	中等	中等
音频点播	低	低	高	低
视频点播	高	低	高	低
电话	低	高	高	低
视频电话	高	高	高	低

图 5-27 应用程序服务质量需求的严格程度

应用程序对它们的带宽需求是不同的，电子邮件、各种形式的音频和远程登录不需要太多的带宽，但文件共享和所有形式的视频应用则需要大量的带宽。

更有趣的是对网络延迟的需求。文件传输应用，包括电子邮件和视频，对延迟并不敏感。如果所有的数据包被均匀地延迟几秒钟，则无伤大雅；交互式应用，比如网上冲浪和远程登录，对延迟比较敏感；而实时应用，例如电话和视频会议，则有严格的延迟要求。如果电话中的每个字都被拖延得太久，那么用户会发现这样的连接是无法令人接受的。另一方面，播放一个服务器上的音频或视频不要求低延迟。

延迟的变化（即标准方差）或者数据包到达时间的变化称为抖动（jitter）。图 5-27 的前三个应用对相邻两个数据包到达时间的无规律性不敏感。远程登录对此有点敏感，如果连接遭遇太多的抖动，屏幕更新将呈现小的突发状。视频，尤其是音频对抖动非常敏感。假设用户正在通过网络观看视频，如果帧都刚好延迟了 2.000 秒，则不会损害播放效果。

但是,如果传输时间随机地在1秒和2秒之间变化,那么结果将非常可怕,除非应用程序能隐藏抖动。对于音频,即使是几毫秒的抖动都能被清楚听到。

相比音视频应用前4种应用对于丢失有更严格的要求,因为任何一位都不允许被错误递交给接收方。实现这个目标的通常做法是由传输层把被网络丢失的数据包重传。这是一项浪费的工作;如果网络一开始就将可能丢失的数据包拒绝或许更好。音视频应用可以容忍少量数据包丢失而无须重传,因为人们不会注意到声音短暂的停顿或者画面偶尔的跳跃。

为了适应各种应用,网络可以支持不同类别的QoS。一个比较有影响的例子是ATM网络。该网络曾经是网络互联宏大远景的一部分,但却早已成为一种利基技术。ATM网络支持:

- (1) 恒定比特率(比如电话)。
- (2) 实时可变比特率(比如压缩的视频会议)。
- (3) 非实时可变比特率(比如点播电影)。
- (4) 可用比特率(比如文件传输)。

这样的分类对于其他的用途或者其他的网络也是有用的。恒定比特率是指试图模拟一条线路,提供恒定的带宽和恒定的延迟。当视频信号被压缩时,由于有些帧的压缩比率大于其他的帧,所以会发生比特率变化的情况。如果发送的帧中包含了很多细节,那么它的发送就要求许多位,而只包含一面白色墙壁的帧可能会被压缩成极少的位。观看点播的电影实际上并不那么实时,因为接收端在开始播放前很容易地就缓冲了好几秒钟的视频,因此网络中的抖动仅仅导致“已存但未播放视频”(stored-but-not-played video)的变化。可用比特率适用于电子邮件这类应用,它们对延迟或抖动并不敏感,而且能做到得到什么样的带宽就用什么样的带宽工作。

5.4.2 流量整形

在网络做出服务质量保证之前,它必须知道要保证哪些流量的服务质量。在电话网络中,这种特性是简单的。例如,语音通话(非压缩格式)需要64 kbps,它由一系列每125微秒的8位样本值组成。然而,数据网络中的流量是突发性的。通常包的到达是非均匀的,比如当流量速率可变(例如,压缩视频会议)、用户与应用程序交互(例如,浏览一个新的网页),以及计算机在不同的任务之间切换。突发流量比固定比特率的流量更难以处理,因为它们可以填满缓冲区并导致数据包丢失。

流量整形(traffic shaping)是指调节进入网络的数据流的平均速率和突发性所采用的技术。它的目标是允许应用程序发送适合它们需求的各种各样流量,包括带有某种程度的突发,但要有一个简单而有用的方式向网络描述可能的流量模式。当一个流在建立时,用户和网络(即客户和网络提供者)就该流的流量模式(即流量的形状)达成一致的协议。实际上,客户可以向服务提供者说“我的流量模型看起来是这样子的;你能处理它吗?”

有时候客户和服务提供者之间的约定称之为**服务等级约定(SLA, service level agreement)**,特别是当它由聚合流量组成,并且存在一段比较长期的时间时,比如一个给定客户的全部流量。只要顾客履行了约定中的义务,并且根据商定的合约发送数据包,那么,服务提供者就承诺按时将数据包递交到目的地。

流量整形技术可以减少拥塞，因此有助于服务提供者兑现它的承诺。然而，要使流量整形工作，还有一个问题需要解决，那就是服务提供商如何确定客户是否遵守它们之间的约定，以及如果客户没有遵守约定，服务提供商该怎么办。超出约定模式之外的数据包可能会被丢弃，或者被打上低优先级标记。对一个流进行监测称为流量监管(traffic policing)。

流量整形和流量监管对于 P2P 和其他可能消耗掉任何可用带宽的数据传输并不那么重要，但它们对实时数据传输有非常重大的影响，比如音频和视频连接等这些具有严格服务质量需求的数据流。

漏桶和令牌桶

我们已经看过一种限制应用程序发送的数据量的方式：滑动窗口，它通过一个参数限制在任何特定时间内可以发送的数据量，因而间接地限制了数据速率。现在我们来描述流量特征的更普遍的方式，漏桶算法和令牌桶流量。方法略有不同，但效果基本均等。请想象这样一个桶，它的底部有一个小洞，如图 5-28 (b) 所示。无论流入漏桶的水的速率多大，只要漏桶中还有水，那么水流出桶的速率是个恒定速率 R ；如果桶内没有水，则流出桶的速率降为 0。此外，一旦桶内的水达到桶的容量 B ，任何额外进入桶的水都会沿着桶侧分流，最终流失。

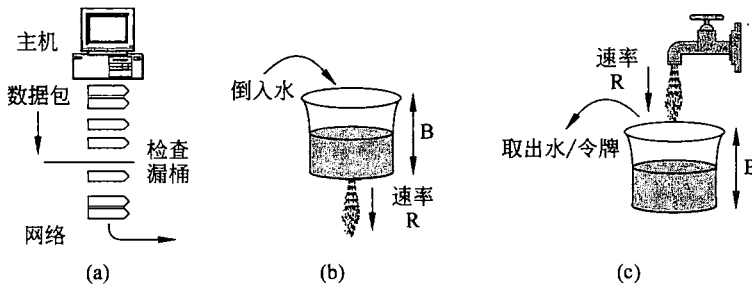


图 5-28

(a) 整形数据包; (b) 漏桶; (c) 令牌桶

漏桶可以应用到注入网络的数据包上，对其进行整形和监管，如图 5-28 (a) 所示。概念上，每个主机在连接到网络的接口中包含一个漏桶。为了向网络发送数据包，必须有可能往漏桶中灌入更多的水。如果漏桶满时来了一个数据包，那么该数据包必须排入队列等漏桶空出来时再接纳，或者被丢弃。前一种策略作为操作系统的一部分可用在对主机进入网络的流量实施整形；后一种策略可用在服务提供商的网络接口，通过硬件对进入网络的流量实施监管。这个技术由 (Turner, 1986) 提出，称为漏桶算法 (leaky bucket algorithm)。

一个形式不同但效果相当的方法是把网络接口想象成一个漏桶，正在往里灌水，如图 5-28 (c) 所示。水龙头速率为 R ，水桶容量为 B (跟以前一样)。现在，为了发送一个数据包，我们必须能够从桶内掏出水或令牌，俗称为内容 (而不是往桶内注水)。桶内只可累积固定数量的令牌，即 B ，不可能有更多的令牌；如果桶是空的，我们必须等更多的令牌到达才能发送另一个数据包。该算法称为令牌桶算法 (token bucket algorithm)。

漏桶和令牌桶限制了一个流的长期速率，但允许其短期突发某个最高调节长度，不会改变延迟也不会受到任何人为的拖延。大量的突发数据将被一个漏桶流量整形器进行平滑处理，以便减少网络拥塞。作为一个例子，假设一个计算机能够产生高达 1000 Mbps 的数

据 (125 万字节/秒), 而且该网络的第一条链路也是以这个速度运行。主机生成的流量模式如图 5-29 (a) 所示。这种模式就是突发性的。每秒的平均速率超过 200 Mbps, 即使该主机以峰值 1000 Mbps 的速率发送 16000 KB 突发数据。

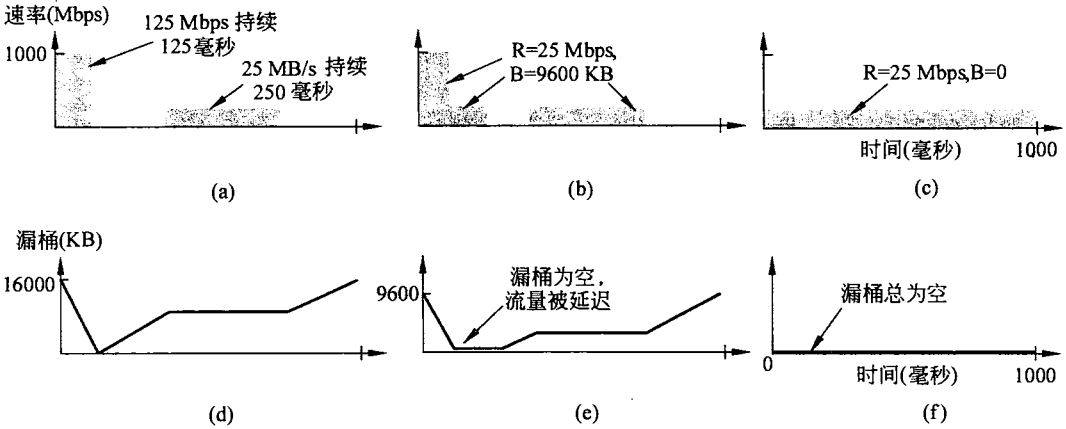


图 5-29

(a) 主机发出的流量。通过一个具有以下容量速率为 200 Mbps 的令牌桶整形输出; (b) 9600 KB 和 (c) 0 KB; 通过一个以下容量速率为 200 Mbps 的令牌桶整形级别: (d) 16 000 KB; (e) 9600 KB; (f) 0 KB

现在假设路由器可以在很短的时间间隔内接受峰值速率的数据, 直到缓冲区填满为止。缓冲区大小为 9600 KB, 远远小于突发流量。在很长的时间内, 路由器的最佳工作速率不能超过 200 Mbps (比方说, 因为这是给予客户的全部带宽)。这里的含义是如果以这种模式发送流量, 其中有一些可能会被网络丢弃, 因为它不适合路由器的缓冲区。

为了避免数据包丢失, 我们可以在主机端用一个令牌桶对其流量进行整形。如果我们使用速率 R 为 200 Mbps, 容量 B 为 9600 KB, 则流量属于网络能够处理的范围。令牌桶的输出显示如图 5-29 (b) 所示。主机可以 1000 Mbps 全速发送一小段时间, 直到它耗尽桶的容量; 然后, 它必须把发送速率削减到 200 Mbps, 直到突发数据被发送出去。其效果是把突发分散在一段时间内, 因为突发量太大无法一次都处理完。该令牌桶的水平如图 5-29 (e) 所示。开始的时候它是全速率发送, 很快被突发耗尽; 当它到达零时, 只能以填满缓冲区的速率发送新的数据包; 此时, 不可能再有突发, 直到桶恢复。没有流量发送时桶被慢慢填满, 当以填充速度发送时桶保持当前水平。

我们也可以把流量整形成更少的突发。图 5-29 (c) 给出了一个以 $R=200$ Mbps 和容量为 0 的令牌桶输出。这是一个极端的案件, 流量被完全平滑了: 不允许任何突发, 并且流量以一个稳定速率进入网络。对应的桶水平如图 5-29 (f) 所示, 桶的水平总是空的。主机上的流量排队等待输出到网络, 并且一旦允许总有一个数据包在等待发送。

最后, 图 5-29 (d) 给出了一个 $R=200$ Mbps, 容量 $B=16\,000$ KB 令牌桶的水平。这是最小的令牌桶, 通过它的流量不会被改变。这个桶可被网络路由器用来监管主机发送的流量。如果主机发送的流量符合它与网络商定的令牌桶, 则流量将通过设置在网络边缘路由器上的相同令牌桶。如果主机以高于最快的或者突发速率发送, 则令牌桶将耗尽。如果发生这种情况, 流量监管器就知道主机发送的实际流量不是事先说明的那样; 然后, 它要么丢弃多余的数据包, 要么降低其优先级, 具体采取何种策略则取决于网络的设计。在我们

的例子中，桶在初始突发结束后被短暂清空，然后恢复到足够下一个突发的水平。

漏桶和令牌桶实现起来很容易。现在，我们描述一个令牌桶的操作。尽管我们已经描述水连续不断地流入和流出漏桶，但真正的实现必须离散度量。令牌桶用一个桶的水平计数器实现。计数器每时钟嘀嗒 ΔT 秒前进 $R/\Delta T$ 单位。在我们上面的例子中是每毫秒为 200 Kb。每次发送一个单位的流量到网络，然后计数器递减；只要计数器非零就一直可以发送。

如果数据包都是一样大小，桶的水平可以数据包计数（例如，200 Kb 对应 20 个大小为 1250 字节的数据包）。然而，通常采用的数据包大小是可变的。在这种情况下，桶的水平就由字节计量。如果剩余的字节数太低，不够发送一个大的数据包，则该数据包必须等待，直到下一个时钟滴答（或者甚至更长的时间，如果填充率很小）。

计算最大突发长度（直到桶清空）需要一点技巧。突发长度刚好要超过 9600 KB 除以 125 KB/s 的除数，因为输出的同时有更多的令牌到达。如果我们称突发长度为 S 秒，最大输出率为 M B/s，令牌桶的容量为 B 字节，令牌到达率为 R B/s，我们可以看到，突发输出最多可包含 $B+RS$ 字节。同时，我们也知道 S 秒时间的最大速度突发长度为 MS 。因此，我们有

$$B + RS = MS$$

解上述等式，得到 $S=B/(M-R)$ 。代入参数 $B=9600$ KB， $M=125$ MB/s， $R=25$ MB/s，我们得到突发时间大约 94 毫秒。

令牌桶算法的一个潜在问题是它把大的突发传输下降到了一个长期速率 R 。人们通常需要把峰值速率降下来，但又不希望降到长期速率（同时也不能提高长期速率让更多的流量流入网络）。平滑流量的一个办法是在第一个令牌桶之后插入第二个令牌桶。第二个桶的速率应比第一个桶高许多。基本上，第一桶表述流量特征，确定其平均速率，但允许少量突发；第二桶降低了进入网络的突发峰值速率。例如，如果第二个令牌桶的速率设定为 500 Mbps，并且容量设置为 0，则初始突发进入网络的峰值速率为 500 Mbps，这比之前的 1000 Mbps 的速率低了许多。

有效使用所有这些桶可能需要不少技巧。当令牌桶被用来整形主机流量时，数据包必须排队并延迟到桶允许它们被发送时；当令牌桶被网络路由器用于流量监管时，该算法确保了发送的数据包不会比允许发送的更多。不过，这些工具提供了把网络流量整形成更易于管理形式的方式，从而协助满足服务质量的要求。

5.4.3 包调度

调整网络流量的形状是保证服务质量的一个良好开端。然而，要想提供性能保证，我们必须沿着数据包经过网络的路径预留足够的资源。为了做到这点，我们假设一个流的数据包都遵循同样的路径。如果这些数据包被随机地分散在路由器上是很难保证什么的。因此，有必要在源端和接收方之间建立起类似虚电路的路径，属于这个流的所有数据包必须遵循这条路由。

在同一个流的数据包之间以及在竞争流之间分配路由器资源的算法称为（数据）包调度算法（packet scheduling algorithms）。为不同的流可以预约的潜在资源有以下 3 种。

- (1) 带宽。
- (2) 缓冲区。
- (3) CPU 周期。

第一种资源“带宽”最为显见。如果一个流要求 1 Mbps，而输出线路的容量为 2 Mbps，那么，试图在这条线路上直接发送 3 个流将不可能正常工作。因此，预留带宽意味着对任何一条输出线路都不能超额预订。

第二种常常短缺的资源是缓冲区空间。当一个数据包抵达时，它通常被保留在路由器的缓冲区直到可以从选择的输出线路上发送出去。当流相互竞争时缓冲区可以起到吸收小的突发流量的作用。如果没有可用的缓冲区，那么该数据包不得不被丢弃，因为没有地方可以存放数据包。对于好的服务质量，可以为某个特定的流预留一定的缓冲区，从而该流不必跟其他的流争用缓冲区。因此，只要该流需要，总能获得可用的缓冲区，直到达到某个最大的限额。

最后，CPU 周期也是一种稀有资源。每个数据包的处理需要占用路由器的 CPU 时间，所以，一台路由器每秒只能处理一定数量的数据包。虽然现代路由器能快速处理大多数数据包，但某些类型的数据包需要得到 CPU 更快的处理，比如我们将在 5.6 节讨论的 ICMP 数据包。为了保证这些数据包及时得到处理，必须确保 CPU 没有过载使用。

数据包调度算法负责分配带宽和其他路由器资源，具体做法是确定下一次把缓冲区中的哪些数据包发送到输出线路。我们在解释路由器工作时已经描述了最直截了当的调度程序。每个路由器把需要转发的数据包排入相应输出线路的队列，直到它们可以发送，并且发送顺序与到达队列的顺序相同。这种算法称为先入先出 (FIFO, First-In First-Out)，或等价的先来先服务 (FCFS, First-Come First-Serve)。

FIFO 路由器在队列满时通常丢弃新到的数据包。由于新到达的数据包会排在队列末尾，因此这种行为称为尾丢弃 (tail drop)。这种处理方式非常直截了当，或许你可能想知道是否还存在其他的处理方式。事实上，我们在 5.3.5 节所描述的 RED 算法在平均队列长度增大时随机选择丢弃一个新到达的数据包。我们将要描述的一些其他调度算法在缓冲区满时也会创造其他的机会来确定丢弃哪些数据包。

FIFO 调度算法易于实现，但它无法提供良好的服务质量，因为当存在多个流时，一个流很容易影响到其他流量的性能。如果第一个流来势汹汹并且发送大的突发数据包，它们将盘踞在队列中。按数据包的到达顺序处理意味着咄咄逼人的发送方能吃掉其数据包穿越的路由器的大部分容量，因而饿死其他流量，降低它们的服务质量。雪上加霜的是想通过路由器的其他流的数据包很有可能被延迟，因为它们不得不排在队列中那个大流量发送者的许多数据包后面。

研究人员已经制定了许多数据包调度算法，这些算法可提供很强的流间隔离，并且能阻止干扰企图 (Bhatti 和 Crowcroft, 2000 年)。其中第一个数据包调度算法是由 (Nagle, 1987) 提出的公平队列 (fair queueing) 算法。该算法的实质是针对每条输出线路，路由器为每个流设置单独的队列。当线路空闲时，路由器循环扫描各个队列，如图 5-30 所示；然后，从下一个队列中取出第一个数据包发送。以这种方式，如果某条输出线路被 n 个主机竞争，则每发送 n 个数据包中每个主机获得发送一个数据包的机会。正是这个意义上的公平，使得所有流量以同样的速率发送数据包。即使源端发送更多的数据包也不会提高这个

速率。



图 5-30 公平队列的循环机制

虽然从一开始，该算法就有一个缺陷：它给使用大数据包的主机比使用小数据包的主机提供了更多的带宽。（Demers 等，1990）建议对该算法的循环策略进行改进，把原来的“数据包接数据包”的循环方式改成“字节接字节”的循环方式。这里的诀窍是计算一个虚拟时间，这个时间指每个数据包发送完毕所需要的轮数。每一轮循环从所有有数据待发送的队列中排空一个字节；然后，按照数据包的结束时间顺序排队，并以该顺序真正发送数据包。

图 5-31 显示了该算法以及该算法的一个例子，例子给出了分别属于三个流的数据包到达和完成时间。如果一个数据包的长度为 L ，它的完成时间恰好是启动之后的第 L 轮循环。启动时间要么是前一个数据包的完成时间，或者是数据包的到达时间（如果它到达时队列为空）。



图 5-31
(a) 加权公平队列； (b) 数据包的完成时间

现在看图 5-32 (b) 中的表，考查最上面两个队列的前两个数据包，数据包的到达顺序是 A、B、D 和 F。数据包 A 在第 0 轮循环到达，长度为 8 个字节，因此其完成时间是第 8 轮；同样，数据包 B 的完成时间为 11；当数据包 B 在发送时数据包 D 到达，因此它的完成时间要从 B 结束时开始算 9 字节循环，最终完成时间为 20。类似地，F 的完成时间为 16。如果没有新的数据包到达，则相对的发送顺序是 A、B、F、D，尽管 F 在 D 之后到达。有可能在最上面的那个流到达另一个很小的数据包，它的完成时间在 D 的完成时间之前。如果 D 的传输尚未开始，那么该小数据包就会跳跃到 D 的前面。公平队列不能抢占当前正在传输的数据包。因为数据包的发送是整体行为，因此公平队列只是理想“字节接字节”方案的近似法。但这是一个很好的近似，任何时候数据包都保持着其理想的传输方案。

实际上这个算法存在一个缺点，即它给所有主机以相同的优先级。在许多情况下，比如，给予视频服务器比文件服务器更多的带宽也是可取的。要做到这点很容易，只要每轮循环时给视频服务器两个或两个以上字节。这种修改后的算法称为加权公平队列（WFQ，

Weighted Fair Queuing)。设每一轮的字节数是一个流的权重 W ，我们现在可以给出计算完成时间的公式：

$$F_i = \max(A_i, F_{i-1}) + L_i / W$$

其中 A_i 为到达时间， F_i 为完成时间， L_i 是数据包 i 的长度。在图 5-31 (a) 中，最下面的队列权重为 2，所以你可以看到在图 5-31 (b) 给出的完成时间表中，它的数据包被发送得更快。

另一个实际的考虑是算法实现的复杂度。WFQ 要求数据包按照它们的完成时间插入到一个有序队列中。如果有 n 个流，则针对每个数据包，至少需要 $O(\log N)$ 操作，这在同时存在许多流的高速路由器上很难实现。(Shreedhar 和 Varghese, 1995) 描述了一种称为赤字循环 (deficit round robin) 的近似算法，该算法的实现非常有效，针对每个数据包只有 $O(1)$ 次操作。WFQ 广泛使用了这个近似算法。

还存在着一些其他类型的调度算法。一个简单的例子是优先级调度，每个数据包被标记一个优先级别。高优先级数据包始终先于任何缓冲的低优先级数据包发送。具有相同优先级的数据包按照 FIFO 顺序发送。但是，优先级调度的缺点是一个高优先级的突发数据包可以饿死低优先级的数据包，这将导致后者无限期地等待下去。WFQ 通常提供了一个更好的选择。通过给高优先级队列更大的权重，例如，高优先级数据包往往会连续发送好几个（因为只有相对较少的数据包具有高优先级），然而，即使存在高优先级的流量，仍然有一定比例的低优先级数据包被陆续发送。一个高低优先级系统基本上是一个双队列的 WFQ 系统，其中高优先具有无限的权重。

数据包调度的最后一个例子是数据包携带时间戳并且按照时间戳顺序发送。(Clark 等, 1992) 描述了一种设计，当数据包被路径上的一系列路由器发送时，时间戳记录该数据包离调度之后或者之前还有多远。排在路由器队列中其他数据包后面的数据包趋向于稍后调度，而首先获得服务的数据包则趋向于优先调度。按数据包的时间戳顺序发送数据包有利于加快慢速数据包，而且同时放缓快速数据包。结果是网络传递的所有数据包具有更一致的延迟。

5.4.4 准入控制

现在我们已经看过了服务质量的所有必要因素，是时候把它们放在一起真正提供服务质量保证了。服务质量的保证通过准入控制的过程来建立。我们首先看的是用于控制拥塞的准入控制，尽管它还很脆弱，但却是性能的保证。我们现在考虑的保障机制更强大，但模型是相同的。用户向网络提供一个有 QoS 需求的流量；然后，网络根据自己的容量以及向其他流做出的承诺决定是否接受或拒绝该流。如果接受，网络就要提前在路由器上预留容量，以便保证新流发送时的服务质量。

沿着数据包经过网络所用的路由，沿途每个路由器都要预留相应的资源。路径上任何一台没有预留的路由器可能变得拥挤不堪，而且单个拥塞的路由器就可打破 QoS 保证。许多路由算法都是在每个源和每个接收方之间找出一条最好路径，并且通过该最好路径发送流量。如果最佳路径上没有足够的剩余容量，这种算法就可能导致某些流遭遇拒绝。如果新流所需要的带宽超过了剩余容量，那么通过选择另外一条产能过剩的不同路径，仍然可能为新流提供 QoS 保证。这就是所谓的 QoS 路由 (QoS routing)。(Chen 和 Nahrstedt, 1998)

对这些技术做了概述。有可能把到达每个目的地的流量拆分到多条路径，以便更容易地发现产能过剩的路径。一个简单的方法是选择同等成本（equal-cost）的路径，并且将流量均等或者按比例分摊到出境链路的容量。然而，更复杂的算法也是可用的（Nelakuditi 和 Zhang，2002）。

给定一条路径，决定接受或拒绝流量并不是一件简单的事情，相对而言，比向一个产能过剩的路由器请求资源（带宽、缓冲区和 CPU 周期）还要稍微复杂一些。首先，尽管某些应用程序可能知道它们的带宽需求，但很少知道有关缓冲区或 CPU 周期的需求，因此至少需要不同的方式来描述流，并且将这种描述转换成路由器资源。我们马上就会看到这点。

其次，某些应用比其他应用更能容忍偶尔错过的最后期限。应用程序必须从网络做出的服务质量保障类型中做出选择，是否硬性保证或者大部分时间都能得到保证。如果所有其他条件都相同，则每个人都希望获得硬性保障，但困难在于它们的代价太昂贵，因为它们限制了最坏情况下的行为。对于应用来说为大多数数据包提供保证往往足够了，而且获得此类保障的更多流可被固定容量支持。

最后，某些应用程序可能愿意就流的参数讨价还价，而其他一些应用可能不会。例如，一个电影观众通常运行速率在 30 帧/秒，如果没有足够的可用带宽来支持 30 帧/秒，他可能愿意把运行速率降到 25 帧/秒。类似地，每帧的像素数、音频带宽等其他特征或许要做出相应的调整。

因为流协商过程中会涉及许多方（包括发送方、接收方，以及发送方和接收方之间沿途的所有路由器），所以必须用特定参数来精确描述流，并且这些参数是可以为各方所协商的。这样的一组参数称为流规范（flow specification）。通常，发送方（比如视频服务器）生成一个流规范，在此流规范中指出它所希望使用的参数；当这份流规范沿着路径传播时，沿途每个路由器对它进行检查，并根据需要修改相应的参数。参数只能往降低质量方面修改，而不能修改成提高流的质量（比如说，修改成低数据率，而不能提高数据率）。当流规范到达另一端的时候，流的参数就被建立起来了。

作为一个关于流规范的例子，考虑如图 5-32 所示的例子。它基于综合服务的 RFC 2210 和 RFC 2211，综合服务是 QoS 的一种设计方案，我们在下一节将对此进行详细的讨论。例子显示的流规范共有 5 个参数。前两个参数“令牌桶速率”和“令牌桶容量”，使用了一个令牌桶，第一个参数给出了发送方可以传输的最大持续速率，即在相当长时间间隔内的平均速率；第二个参数给出了短时间间隔内可以发送的最大突发量。

第三个参数，峰值速率指网络能容忍的最大传输速率，即使在很短的时间间隔内也要受此约束。发送方在短时间内突发传输时也不得超过这个速率。

最后两个参数指定了数据包的最小和最大长度，包括传输层和网络层的头（比如 TCP 和 IP）。最小长度的规定是有用的，因为不管一个数据包有多短，在对它进行处理时需要一些固定时间。一台路由器可能每秒钟能够处理 10 000 个 1 KB 长度的数据包，但是可能无法处理 100 000 个 50 字节长度的数据包，尽管后者的数据率比前者更低一些。最大数据包长度很重要，因为可能会存在一些无法超越的内部网络限制。例如，如果路径上有一段

参数	单位
令牌桶速率	字节/秒
令牌桶容量	字节
峰值速率	字节/秒
最小数据包尺寸	字节
最大数据包尺寸	字节

图 5-32 流规范实例

是以太网，那么，不管其他网络如何处理数据包，数据包的最大长度将被限制于不得超过 1500 字节。

一个有趣的问题是路由器如何将一个流规范转变成一组特定的资源而进行预留。乍一看，似乎很明显，假设一个路由器有条链路运行速率为 1Gbps，并且平均包的长度为 1000 位，那么它能以每秒一百万个包的速率处理数据包。实际情况并非如此，由于负载的统计波动，链路总是会有空闲周期。如果链接需要利用容量的每一位来完成自己的工作，那么甚至数位的空转都可能生成一个永远也无法摆脱的积压。

即使实际负载略微轻于理论容量，也可能排起队列，而且延迟仍然可能会发生。请考虑这样的情形：数据包随机到达，平均到达率为每秒 λ 个数据包；数据包具有随机长度，并且可以平均速率每秒 μ 个数据包发送到链路上。假设数据包的到达和服务均服从泊松 (Poisson) 分布（这就是所谓的 M/M/1 排队系统，其中 M 指 Markov，即 Poisson），则利用排队理论可以证明，一个数据包所经历的平均延迟 T 为：

$$T = \frac{1}{\mu} \times \frac{1}{1 - \lambda/\mu} = \frac{1}{\mu} \times \frac{1}{1 - \rho}$$

这里 $\rho = \lambda/\mu$ 是 CPU 的利用率。第一个因子 $1/\mu$ 是在没有竞争情况下的服务时间。第二个因子是指由于跟其他流竞争而导致的减慢因素。例如，如果 $\lambda = 950\,000$ 数据包/秒， $\mu = 1\,000\,000$ 数据包/秒，那么， $\rho = 0.95$ ，并且每个数据包经历的平均延迟将是 20 微秒，而不是 1 微秒。这个时间值包含了排队时间和服务时间，正如当负载很低时（即 $\lambda/\mu \approx 0$ ）能看出这一点。如果在流的路径上有 30 台路由器，那么仅仅排队延迟就会达到 600 微秒。

(Parekh 和 Gallagher, 1993, 1994) 给出了一个对应于带宽和延迟性能保证的流规格说明方法。该方法在流量源端采用 (R, B) 令牌桶整形，而在路由器采用 WFQ。每个流都有一个 WFQ 权重 W ，该权重值足够大到能排空速率为 R 的令牌桶，如图 5-33 所示。例如，如果流的速率为 1 Mbps，路由器和输出链路的容量为 1 Gbps，那么在路由器的输出链路上该流的权重必须大于所有流的全部权重的千分之一。这保证了该流具有最低带宽。如果流得不到足够大的速度，那么该流就不允许进入网络。

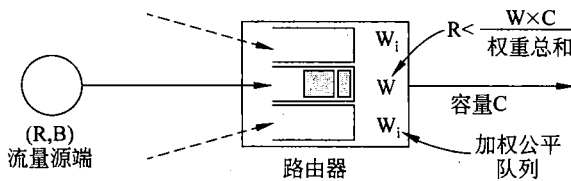


图 5-33 采用令牌桶和 WFQ 的带宽和延迟保证

流的最大排队延迟是令牌桶突发大小的函数。考虑两个极端情况。如果流量是平缓的，没有任何突发，数据包将以它们到达的速率被路由器排空，此时将不会有排队延迟（忽略打包效果）。另一方面，如果流量被保存起来形成堆积，那么一次到达路由器的最大突发数为 B 。在这种情况下，最大排队延迟 D 将是以保证带宽排空突发的时间，或 B/R （再次，忽视打包效果）。如果这个延迟过大，那么流必须向网络请求更多的带宽。

这些保障是硬性的。令牌桶约束了源端的突发性，公平队列隔离了给予不同流的带宽。这意味着无论其他竞争对手在路由器上的行为如何，该流将得到其带宽和延迟保证。那些其他流量不能破坏这种保障，即使积累流量并一次全部发送出来也不会影响到上述流量的

服务质量。

此外,这个结果对任何网络拓扑结构下通过多个路由器的路径都成立。每个流得到了最低带宽,因为这是每个路由器做出的带宽保证。每个流得到最大延迟的理由则有点微妙。在最坏的情况下,冲击第一个路由器的突发流量与其他流的流量展开竞争,它将被推迟到最大延迟 D 。然而,这种延迟能平滑突发流量。反过来,这意味着这次突发将再也不会后面的路由器遭受进一步的排队延迟。总的排队延迟最大为 D 。

5.4.5 综合服务

从 1995 年到 1997 年之间, IETF 做了很大的努力来设计流式多媒体的体系结构。这项工作最后产生了 20 多个 RFC, 从 RFC 2205~2210。此项工作的通用名称是综合服务(integrated service), 主要针对单播和组播应用。比如说, 用户从一个新闻网站抓取一段视频片段就是单播应用的一个例子; 一组数字电视台将它们的节目以 IP 数据包流的方式广播到各地的许多接收端, 就是组播应用的一个例子。下面我们将重点关注组播, 因为单播可以看作是组播的一个特例。

在许多组播应用中, 组的成员可能会动态地发生变化。例如, 用户进入一个视频会议, 然后感觉厌烦了就切换到一个肥皂剧或者橄榄球频道。在这样的情况下, 让发送方提前预留带宽的做法并不能很好地工作, 因为这要求每个发送方必须跟踪它的听众们的全部加入和离开情况。对于一个拥有上百万个用户的电视传输系统而言, 这样的设计显然不能工作。

RSVP——资源预留协议

在综合服务体系结构中, 最主要的也是网络用户可见的那部分是资源预留协议(RSVP, Resource reSerVation Protocol)。RFC 2205~2210 文档对该协议进行了详细描述。该协议的主要功能是预留资源, 发送数据则需要使用其他协议。RSVP 允许多个发送方给多个接收组传送数据, 也允许接收方自由地切换频道, 并且在消除拥塞的同时优化带宽的使用。

在最简单的形式下, RSVP 使用了基于生成树的组播路由, 关于用生成树来实现组播路由前面已经有所讨论。每个组都分配一个组地址。为了给一个组发送数据包, 发送方将该组的地址放到这些数据包中。然后, 标准的组播路由算法建立起一棵覆盖所有组成员的生成树。路由算法并不是 RSVP 的一部分。与常规组播的唯一不同之处是这里的组播需要一些额外的信息, 这些信息被周期性地组播给生成树中的路由器, 告诉它们在内存中维护特定的数据结构。

举例来说, 请考虑图 5-34 (a) 中的网络。主机 1 和主机 2 是组播发送方, 主机 3、4 和 5 是组播接收方。在这个例子中, 发送方和接收方是分离的; 但是一般情况下, 这两个集合可以重叠。针对主机 1 和主机 2 的组播树分别如图 5-34 (b) 和 (c) 所示。

为了获得更好的接收效果并且消除拥塞, 一个组中的任何接收方都可以沿着树给组播发送方发送一条预留消息。利用前面讨论过的逆向路径转发算法, 该预留消息被传播到发送方。在沿途的每一跳, 路由器会注意到此预留消息, 并预留必要的带宽。在前一节中我们已经了解到如何使用加权公平队列包调度算法来进行资源预留。如果没有足够的带宽, 它就返回报告失败。当这条预留消息到达组播发送方时, 从发送方到该接收方一路上的带宽都得到预留, 因为沿着生成树完成了预留。

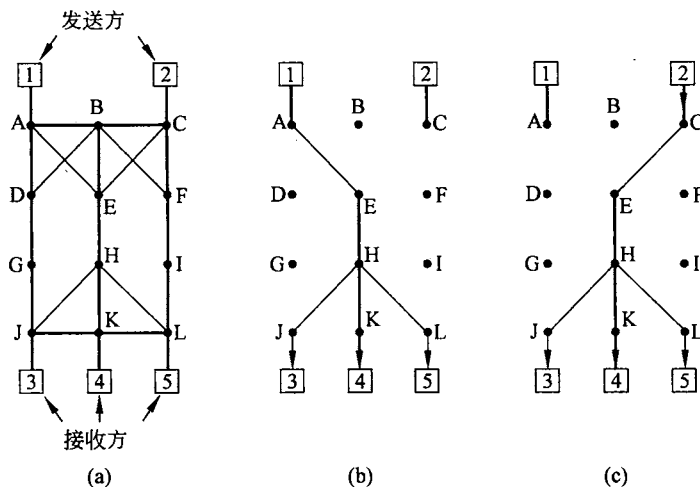


图 5-34

(a) 一个网络；(b) 主机 1 的组播生成树；(c) 主机 2 的组播生成树

图 5-35 (a) 显示了一个这种资源预留的例子。在这里，主机 3 请求一条通向主机 1 的信道。一旦该信道被建立起来，则从主机 1 到主机 3 的数据包流将不会再遭遇拥塞。现在请考虑，如果接下来主机 3 为了能同时观看两套电视节目，要预留一条通向另一个发送方（即主机 2）的信道，会怎么样？第二条预留的路径如图 5-35 (b) 所示。请注意，从主机 3 到路由器 E 之间需要两条独立的信道，因为传输的是两个独立的流。

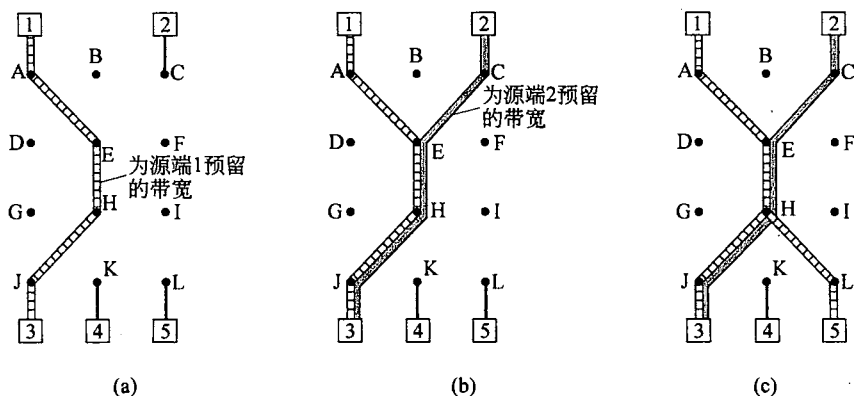


图 5-35

(a) 主机 3 向主机 1 请求一条信道；(b) 然后主机 3 向主机 2 请求第二条信道；(c) 主机 5 向主机 1 请求一条信道

最后，在图 5-35 (c) 中，主机 5 决定观看主机 1 传送的节目，因而也请求预留带宽。首先，在主机 5 到路由器 H 之间，要预留专门的带宽。然而，路由器 H 看到自己已经有了一份来自主机 1 的流，所以既然所需的带宽已经被预留了，就没有必要再次预留。请注意，主机 3 和主机 5 请求的带宽数量可能会不相同（比如，主机 3 在小屏幕上播放节目，只需要低分辨率信息），所以，预留的带宽数量必须足够大，才能满足最贪婪的那个接收方。

在进行资源预留时，接收方可以（有选择地）指定一个或者多个期望接收的数据源。它也可以说明在预留期间这些选择是否固定不变，或者是否希望以后还可以改变数据源。路由器利用这些信息来优化带宽的使用计划。尤其是，如果两个接收方都同意以后不再改

变数据源的话，那么它们只需共享一条路径即可。

采用这种完全动态策略的理由是将被预留的带宽与数据源的选择分离开。一旦接收方已经预留了带宽，那么它可以切换到另一个数据源，保留现有路径上那部分带宽并且对新数据源仍然有效。例如，如果主机 2 正在实况传输几个视频流，比如这是一个拥有多个节目频道的 TV 电视台，那么主机 3 可以随意地在这几个频道之间切换，而根本不用改变它所做的预留：路由器并不关心接收方正在观看什么节目。

5.4.6 区分服务

基于流的算法有能力为一个或者多个流提供非常好的服务质量，因为它们在沿途路由上预留了必要的资源。然而，这些算法有一个缺点，它们都需要为每个流预先进行设置。当存在数千或数百万个流时，这些算法就不能很好地扩展使用；而且，在路由器中为每个流维护一个内部状态很容易导致路由器的崩溃；最后，为了设置数据流，需要修改的路由器代码量很大，并且还涉及复杂的路由器-路由器之间消息交换。而且，尽管综合服务方面的研究工作还在继续并往前推进，但 RSVP 几乎没有实际部署，甚至类似的实现也很少。

基于这些原因，IETF 还设计了另一个更加简单的服务质量方法，该方法很大程度上由每个路由器本地实现，无须事先提前设置流，也不牵涉整条路径。这种方法称为基于类别（class based）的服务质量（相对于基于流的服务质量），还有另一个更流行的名称为区分服务（differentiated service）。IETF 已经对该方法的体系结构进行了标准化，RFC 2474、2475 和其他一些 RFC 文档对区分服务进行了详细描述。下面我们介绍区分服务。

区分服务可以由一组路由器提供，这些路由器构成了一个管理域（比如一个 ISP 或者一家电话公司）。管理规范定义了一组服务类别，每个服务类别对应于特定的转发规则。如果一个客户已经订购了区分服务，那么进入到该管理域的客户数据包就会被标上它们属于哪类服务。这个信息可由 IPv4 和 IPv6 数据包（在 5.6 节讨论）的区分服务字段携带。服务类别定义为单跳行为（PHP, per hop behaviors），对应于数据包在每个路由器得到的待遇，而不是对数据包在整个网络中的保证。具有某种单跳行为（比如优质服务）的数据包相比其他数据包（比如普通服务）可以获得更好的服务。属于同一个类别的通信流量可能要先经过处理以便符合特定的形状特征，比如通过一个具有特定排空速率的漏桶。商业嗅觉灵敏的运营商可能对每个优质服务类别的数据包收取额外的费用，或者每个月收取固定的额外费用并允许最多 N 个优质服务类别的数据包。请注意，这种方案并不要求提前设置，也没有资源预留，更不需要花时间为每个流进行端到端的协商。这些服务特征与综合服务有很大的不同，也使得区分服务相对容易实现。

基于类别的服务在其他工业领域也有。例如，包裹托运公司通常提供昼夜送达、两天内送达和三天内送达等不同的服务；航空公司提供头等舱、商务舱和经济舱服务；长途列车一般也有多个服务等级。甚至巴黎的地铁还有两种服务类别。对数据包而言，类别之间的差异可以体现在延迟、抖动、发生拥塞时被丢弃的概率，以及其他的可能性（如果不是以太帧的话，还有其他一些可能）。

为了更好地理解基于流的服务质量与基于类别的服务质量之间的差异，请考虑一个实例：Internet 电话。如果采用基于流的方案，每个电话呼叫都拥有它自己专用的资源，从而

服务质量可以保证。如果采用基于类别的方案,则所有的电话呼叫合起来预留同一份预留给电话类别的资源。这些资源不能被网页浏览类别或者其他类别的数据包夺走,但是,任何一个电话呼叫也得不到任何独自享用的私有资源。

加速转发

服务类别的选择取决于每个运营商,但是由于通常情况下大多数数据包需要在不同运营商的网络之间转发,所以,IETF 已经定义了某些与网络无关的服务类别。最简单的服务类别是加速转发(expedited forwarding),RFC 3246 对此有详细的描述。我们先从这个种类别开始讨论。

加速转发背后的思想非常简单。我们可以把服务类别分为两种:常规的和加速的。绝大多数通信流量属于常规流量,但是有一小部分数据包需要加速转发。加速类别的数据包应该可以直接通过网络,就好像不存在其他任何数据包一样。这样,它们将获得低丢失、低延迟和低抖动服务——这正是 VoIP 所需要的服务。这种“双管道”系统的一种符号化表示方法如图 5-36 所示。请注意,这里依然只有一条物理线路,图中的两根逻辑管道只是表示了为不同类别服务预留带宽的方法,而不是有两条物理线路。

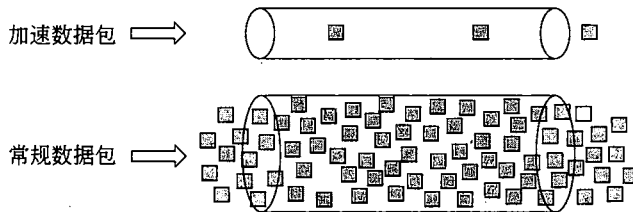


图 5-36 加速数据包体验一个无流量网络

实现这种策略的一种做法如下:把数据包分类成加速和常规两种,并做相应的标记。这一步可以在发送主机上完成,或者在入口(第一个)路由器上做。在发送主机上进行分类的好处是它可以利用更多的信息来确定哪些数据包属于哪些流。这个任务可以由网络软件甚至操作系统来执行,以免更改现有的应用程序。例如,主机为 VoIP 数据包标记上加速服务标记的做法正在逐步流行起来。如果这些数据包穿越支持加速服务的公司网络或 ISP,它们将获得优惠待遇;如果网络不支持加急服务,也没有什么坏处。

当然,如果标记工作由主机来做,则入口路由器很可能就承担着监管流量的任务,确保客户没有发送比他们所付费用更多的加速流量。在网络内部,路由器针对每条出境线路可以有两个输出队列,一个用于加速数据包,另一个用于常规数据包。当一个包到达时,它被排入相应的队列。加速队列获得的优先级要高于常规队列,例如,使用优先级数据包调度算法。通过这种方式,加速数据包看到的是一个没有负载的网络,即使事实上普通流量的负载很重。

确保转发

管理服务类别的一种更精细方案称为确保转发(assured forwarding)。确保转发服务由 RFC 2597 定义,它规定了 4 种优先级别,每种级别都拥有自己的资源。前 3 种服务类别或许可分别称为金质、银质和铜质。而且,标准还针对正经历拥塞的数据包,定义了 3 种丢弃概率:低、中、高。将这两个因素结合起来,共有 12 种服务类别。

图 5-37 显示了一种确保转发下的数据包处理办法。第一步是将数据包分成 4 个优先级别之一。如前所述，这一步有可能在发送主机上完成（如图中所示），也可能在入口路由器上完成，高优先级数据包的速率可作为服务供给的一部分由运营商限制。

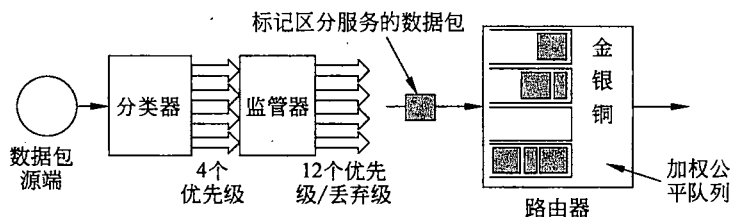


图 5-37 确保转发的一种可能实现

下一步是确定每个数据包的丢包类别。每个优先级的数据包穿过一个诸如令牌桶这样的流量监管器，可以完成这个工作。该监管器让所有的流量都通过，但它根据突发流量大小来标识数据包的丢包概率：符合小突发量的标为低概率、大于小突发量的标为中概率、超过大突发量的标为高概率。然后将优先级和丢包概率结合起来，编码到每个数据包中。

最后，数据包由网络内部的路由器处理，路由器上的数据包调度器区分不同类别的数据包。一般的选择是针对 4 个优先级类别采用加权公平队列，给予较高的类别以较高的权重。在这种方式中，高优先类别的数据包将获得大部分的带宽，但较低优先类别的数据包也不会完全被饿死。例如，如果一个类别的权重是低一级类别权重的两倍，则高类别的数据包将获得两倍于低类别数据包的带宽。在同类别的优先级内部，具有较高丢包概率的数据包可被运行的算法优先丢弃，比如我们在 5.3.5 节了解过的 RED 算法（随机早期检测）。当刚刚出现拥塞苗头且路由器还有缓冲空间时，RED 就开始丢弃数据包。在这个阶段，路由器仍然有缓冲空间用来接受低丢包概率的数据包，而丢弃高丢包概率的数据包。

5.5 网络互联

到现在为止，我们一直隐含着假设所讨论的网络是一个同质网络，即每台机器在每一层上使用同样的协议。不幸的是，这样的假设太过于乐观。实际上，存在着许多不同的网络，包括 PAN、LAN、MAN 和 WAN。我们已经描述了以太网、线缆上的 Internet、固定和移动电话网络、802.11、802.16 等。大量协议被广泛应用于这些网络的各个层次。在下面的章节中，我们将详细讨论当两个或者多个网络连接起来形成网络互联（internetwork），或简单的互联网（internet）时所涉及的一些问题。

如果每个人都使用单一的网络技术，那么将网络连接起来非常简单，而且经常会出现某种类型的网络占据主导位置，比如以太网。一些专家推测技术的多样性会自动消失，只要每一个人都意识到 [填入你最喜爱的网络] 是多么的美妙。但是，不要指望这种情况会发生，历史证明这是一相情愿的想法。不同类型的网络解决不同的问题，因此以太网和卫星网络可能永远都不会相同。重用现有的系统增加了限制，比如在线缆之上、电话网络之上和电源线之上运行数据网络，从而造成网络特性的发散和分叉。这就是异质性的由来。

如果总会有不同的网络，如果我们不需要互连它们，事情更简单。显然，这也是不可

能的。Bob Metcalfe 推测一个具有 N 个节点的网络的價值等于节点之间的连接数，或者 N^2 (Gilder, 1993)。这意味着大型网络比小型网络更有价值，因为它们允许更多的连接，所以始终有将小型网络联结起来的激励。

Internet 是这种互连 (interconnection) 的最佳例子 (我们将把 Internet 的首字母写成大写的 “I” 以示和其他互联网或者连接在一起的网络区别开来)。纳入所有这些网络的目的是使得任何一种网络的用户都可以和其他种类的网络用户沟通。当你向 ISP 支付 Internet 服务费用时，收取的费用取决于你的线路带宽，但你真正支付的是能够与同样连接到 Internet 上的其他主机交流数据包的能力。毕竟，如果你只能将数据包发送到同一城市的其他主机，Internet 就不会像现在这样受欢迎。

由于网络往往在一些重要方面有所不同，因此一个网络得到来自另一个网络的数据包并不总是那么容易。我们必须解决异质性的问题，以及因相互连接起来的互联网增长非常大而造成的规模问题。我们首先考查不同的网络是如何的不同，以便寻找到相应的解决方法；然后，我们将学习 Internet 网络层协议 IP (Internet 协议) 获得成功的经验，包括穿越网络的隧道、互联网络的路由和数据包拆分技术。

5.5.1 网络如何不同

网络的不同可体现在不同方面。比如不同的调制解调技术或帧格式这样的差异属于物理层和数据链路层内部，这些差异我们在这里不关心。相反，在图 5-38 我们列出了暴露在网络层面的一些差异。正是这些被掩盖的差异使得对互联网络的操作比对单个网络操作更加困难。

项目	某些可能性
提供的服务	无连接与面向连接
寻址	不同大小，扁平或层次
广播	提供或者缺乏(组播同样)
数据包尺寸	每个网络有自己的最大尺寸
有序性	有序和无序传递
服务质量	提供或缺乏；许多不同种类
可靠性	丢包的不同级别
安全性	隐私规则，加密等
参数	不同超时值，流规范等
记账	按连接时间、包数、字节数或不收费

图 5-38 网络的某些不同之处

当一个网络上的某个源端发出的数据包必须要经过一个或者多个外部网络才能到达目标网络时，网络之间的接口可能会产生许多问题。首先，源端必须能够寻址接收方。如果源端在以太网络上，而接收方在 WiMAX 网络上，我们应该做什么？假设我们可以在以太网标识一个 WiMAX 目标，数据包将从一个无连接网络穿越到面向连接的网络，可能需要在短时间内建立一个新的连接，这将带来延迟，而且，如果连接不是被更多的数据包使用，开销很大。

许多特殊的分歧也必须能够被容纳。我们如何在一个不支持组播的网络上把数据包分发给一组用户？不同网络规定的最大数据包尺寸不同，这也是困扰网络互联的主要因素。如何通过一个最大尺寸为 1500 字节的网络传递长度为 8000 个字节的数据包？如果一个面向连接网络上的数据包经过一个无连接网络，它们可能以不同于发送顺序到达接收方。这可能是发送方没有想到的，因而可能会引起接收方的（不愉快的）诧异。

经过一些努力，这类差异可以被掩盖掉。例如，连接两个网络的网关可以为每个接收方生成单独的数据包，来替代支持组播的更好网络；一个大的数据包可能被拆分，分段发送，然后再重组还原。接收端缓冲收到的数据包，并按顺序递交它们。

网络还可能在其他大的方面有所区别，而这些方面是难以调和的。最明显的例子是服务质量。如果一个网络具有强大的 QoS，而其他网络只提供尽力而为的服务，那么就不可能为端-端的实时流量做带宽和延迟保证。事实上，除非运行的尽力而为网络利用率较低，否则服务质量可能只是说说而已很难真正采用，因此这不大可能成为大多数 ISP 的目标。安全机制也有问题，但至少可以在不具备安全性的网络顶端设置加密技术，来保证保密性和数据完整性。最后，当平时正常使用的网络突然变得昂贵起来，计费上面的差异可能会产生令人不悦的账单，正如有数据计划的漫游手机用户所发现的那样。

5.5.2 何以连接网络

连接不同网络的方式有两种基本选择：第一，我们可以制造这样的设备，它能将每种网络的数据包翻译或转换成每个其他类别网络的数据包；第二，像出色的计算机科学家那样，尝试在不同网络的上面增加一个间接层，并且构造一个公共层来解决这个问题。这两种情况下，新设备被放置在网络之间的边界上。

早期，(Cerf 和 Kahn, 1974) 提出用一个公共层来隐藏现有网络的差异。这种方法已经取得了巨大成功，他们提出的层最终被分别融入到 TCP 和 IP 协议。差不多 40 年之后，IP 成为了现代 Internet 的基础。由于这个成绩，Cerf 和 Kahn 于 2004 年被授予图灵奖，该奖相当于计算机科学领域的诺贝尔奖。IP 提供了一种通用的数据包格式，所有路由器都认识这种数据包，因而这种数据包几乎可以通过所有的网络传递。IP 已经将其研究活动从计算机网络扩展到电话网络。它还可以运行在传感器网络和其他微型设备上，这些微小设备一度被认定资源太少而无法支持 IP。

我们已经讨论了几种用来连接网络的不同设备，包括中继器、集线器、交换机、网桥，路由器和网关。中继器和集线器只是将比特从一根导线移动到另一根导线，它们大多是模拟设备，不了解有关高层协议的任何知识。网桥和交换机工作在链路层。它们可以被用来构建网络，但只能处理轻微的协议转换，例如，在 10、100 和 1000 Mbps 以太网交换机之间传递帧。我们在这一节的重点是相互连接工作在网络层的设备，即路由器，把网关等高层互连设备留到以后介绍。

让我们先探讨在较高的层次如何用一個共同的网络层来互连不同的网络。一个由 802.11、MPLS 和以太网网络组成的互联网络，如图 5-39 (a) 所示。假设源主机在 802.11 网络上，要给以太网上的目标机器发送数据包。由于这两个网络技术不同，而且它们又被另一类型的网络 (MPLS) 隔离，因此在网络之间的边界需要做一些额外的处理。

因为，一般来说不同的网络有不同形式的地址，数据包携带一个网络层地址，它可以标示这三个网络上的主机。当一个数据包从 802.11 网络被发送到 MPLS 网络时，首先到达第一个网络边界。802.11 提供了无连接服务，但 MPLS 提供了面向连接的服务。这意味着，必须建立一条穿过该网络的虚电路。只要数据包沿此虚电路传输，就能到达以太网网络。在第二个网络边界，数据包可能太大以至于无法通过，因为 802.11 可以使用比以太网大得多的帧。为了解决这个问题，数据包被拆分为段，每个段单独发送。当这些段达到接收方后，它们被重新组合在一起。这时，数据包才完成自己的旅程。

这个过程当中的协议处理如图 5-39 (b) 所示。源端接受来自传输层的数据，并生成一个带有公共网络层的头，这个例子中采用的是 IP 协议。网络层的头包含了最终的接收方地址，这个地址被用来确定数据包应该通过第一个路由器发送。因此，包被封装在 802.11 帧内并且被发送出去，帧的目标地址是第一个路由器的地址。在路由器，数据包从帧的数据字段中被提取出来，802.11 帧头被丢弃；现在路由器检查数据包中的 IP 地址，并查询其路由表；根据这个地址，路由器决定将数据包发送到第二个路由器。对于路径中的这部分，路由器必须建立一条到第二个路由器的 MPLS 虚电路，并且必须用 MPLS 报头封装该数据包。在远端，MPLS 头被丢弃，并在此检查网络地址，以便寻找下一跳网络层。这下一跳就是目标网络本身。由于数据包太长无法通过以太网发送，它被拆分为两部分。每一部分被放入以太网帧的数据字段，并被发送到目的地的以太网地址。在接收方，剥掉每个帧的以太网帧头，将帧的数据内容重新组合。至此，数据包终于到达了它的目的地。

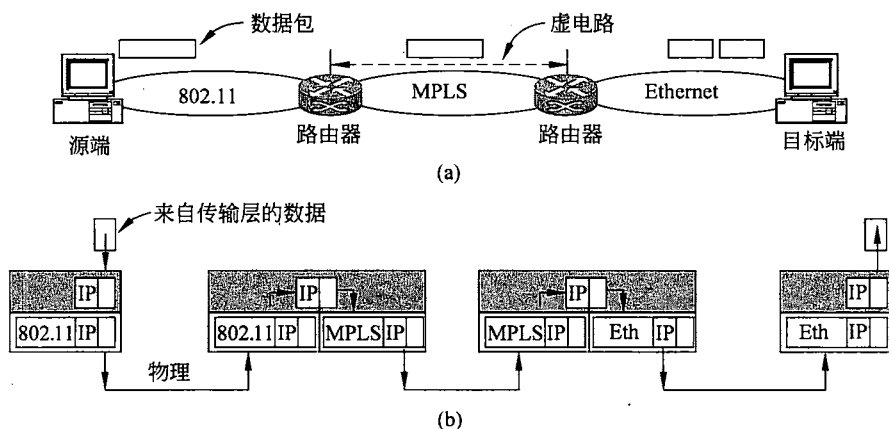


图 5-39

(a) 跨越不同网络的数据包; (b) 网络层和链路层的协议处理

这里可以观察到路由情况和交换（或桥接）情况的本质区别。在路由器上，数据包被从帧中提取出来，数据包中的网络地址被用来决定把它转发到哪里；而在交换机（或网桥）上，整个帧是根据其 MAC 地址传送的。交换机不必了解正在被交换的数据包所采用的网络层协议，同样路由器也不必了解交换机所采用的链路层协议。

不幸的是，网络互联并不像我们说的那么容易。事实上，引入网桥的目的就是用它们将不同类型的网络联结起来，或者至少把不同类型的局域网联结在一起。它们的做法是把一个 LAN 的帧翻译成另一个 LAN 的帧。然而，这样的工作并不理想，出于同样的原因网络互联是很困难的：LAN 特征上的差异很难掩盖，比如不同的最大数据包尺寸、LAN 有

优先级或者没有。今天，网桥主要用来连接链路层的同类网络，路由器用来连接网络层不同的网络。

网络互联技术在建设大型网络方面非常成功，但仅当有一个共同网络层时才能工作。事实上，随着时间的推移已经存在许多网络协议。当公司认定有一个自己控制的专用格式是它们的商业优势时，想要获得大家对单一格式的一致认可就很困难了。除了 IP 这个例外，现在近乎普遍的网络协议是 IPX、SNA 和 AppleTalk，这些协议仍然没有得到广泛使用，但总是会有其他协议的。现在最相关的例子可能是 IPv4 和 IPv6。虽然这些协议是 IP 的两个版本，但它们是不兼容的（否则就没有必要建立 IPv6 了）。

可以处理多个网络协议的路由器称为多协议路由器（multiprotocol router）。它必须翻译协议，或者把连接留给更高的协议层。这两种方法都不完全令人满意。更高层的连接，比如说 TCP，要求所有的网络都实现 TCP（也许并非如此）。因此，它限制网络只能被使用 TCP 的应用使用（不包括许多实时应用）。

另一种方法是在网络之间转换数据包。然而，除非数据包格式比较相近，具有相同的信息字段，否则这种转换将永远是不完整的，并且往往注定要失败。例如，IPv6 地址为 128 位长，不管路由器如何努力尝试，它们肯定不适合填入 32 位的 IPv4 地址字段。在一个网络上同时运行 IPv4 和 IPv6 已被证明是部署 IPv6 的一个主要障碍（为公平起见，首先要让客户明白为什么他们应该使用 IPv6）。在两个根本不同的协议之间进行翻译可能产生更大的问题，比如无连接和面向连接的网络协议。鉴于这些困难，很少有人去尝试这种转换。可以说，甚至 IP 也只能在作为最小分母的一种服务时才工作得很好。IP 对运行自己的网络要求很少，但它提供的只能是尽力而为的服务。

5.5.3 隧道

处理两个不同网络相互连接时的一般情况超级困难。然而，却存在一种最常见的甚至可管理不同网络协议的情形。这种情形就是源主机和目标主机所在网络的类型完全相同，但它们中间却隔着一个不同类型的网络。举例来说，请考虑一家跨国银行，它在巴黎有一个 IPv6 网，在伦敦也有一个 IPv6 网，但是连接巴黎和伦敦办事处的却是 IPv4 Internet，图 5-40 显示了这样的情形。

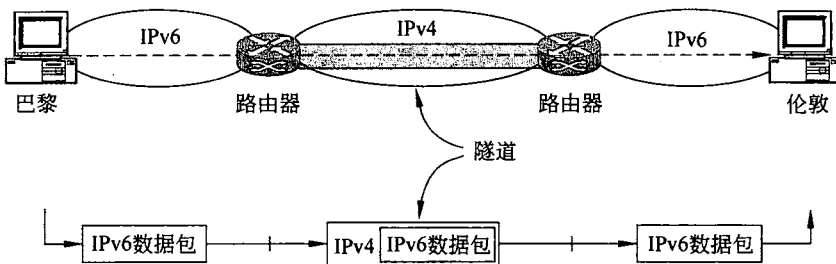


图 5-40 从巴黎隧道一个数据包到伦敦

这个问题的解决方案是一种称为隧道（tunneling）的技术。为了给伦敦办事处的主机发送一个 IP 数据包，巴黎的主机构造一个包含伦敦 IPv6 地址的数据包，然后将该数据包发送到连接巴黎 IPv6 网络到 IPv4 Internet 上的多协议路由器；当该多协议路由器获得 IPv6

数据包后，它把该数据包用一个 IPv4 头封装，封装后的 IPv4 数据包指向多协议路由器另一边的 IPv4，该网络与伦敦的 IPv6 网络相连；也就是说，路由器将一个（IPv6）数据包放入到一个（IPv4）数据包中。当这个包裹着的数据包到达伦敦路由器，原来的 IPv6 数据包被提取出来，并被发送给最终的目标主机。

可以把通过 IPv4 Internet 的路径看作是一根从一个多协议路由器延伸到另一个多协议路由器的大隧道。IPv6 数据包只是从隧道的一端旅行到隧道的另一端，封装在漂亮的盒子里非常舒适。它全然不必担心与 IPv4 的相处事宜。巴黎和伦敦的主机也不需要担心任何有关 IPv4 的事情。只有多协议路由器必须了解 IPv4 和 IPv6 数据包。实际上，从一个多协议路由器到另一个多协议路由器的整个行程就像单条链路上的一跳。

用一个类比可能会使隧道的概念更加清晰。考虑一个人驾着一辆汽车从巴黎出发要去伦敦。在法国境内，该汽车可以依靠自己的马力向前行驶，但是当它到达英吉利海峡，它被装到高速列车中，经过海底铁路隧道到达英国（汽车不允许直接在隧道中行驶）。实际上，这里的汽车被当作货物一样运到另一端，如图 5-41 所示。到了隧道另一端，汽车被放在英国公路上，它又可以依靠自己的马力向前行驶了。数据包在通过外部网络时所使用的隧道技术也是以同样的方式工作。

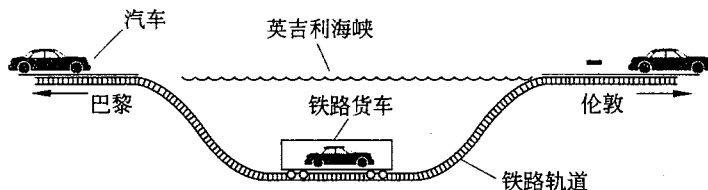


图 5-41 通过隧道把一辆车从法国运到英国

隧道被广泛用于连接那些因使用其他网络而被隔离的主机和网络。结果生成的网络就是所谓的覆盖（overlay）网络，因为它有效地覆盖在基础网络之上。部署一个具有新特性的网络协议是采用隧道的一个共同原因，就像我们说明的“IPv4 之上的 IPv6”例子。隧道的缺点是无法到达位于隧道之下网络的主机，因为数据包无法从隧道中间逃生。

然而，隧道的这个限制变成了虚拟专用网络（VPN，Virtual Private Networks）的优势。VPN 就是一个提供安全措施的简单覆盖网络。我们将在第 8 章探讨 VPN。

5.5.4 互联网路由

通过互联网的路由所面临的基本问题与单个网络中路由的基本问题相同，但复杂性有所增加。首先，内部网络可能使用不同的路由算法。例如，一个网络可以使用链接状态路由，而另一个网络使用了距离矢量路由。由于链路状态算法需要知道拓扑但距离矢量算法不需要，仅仅这个差异就导致两个网络都不清楚如何在互联的网络上找到最短路径。

网络由不同运营商运行带来了更大的问题。首先，运营商对于什么是通过网络的好路径有不同的想法。一个运营商可能希望用最少延迟的路由，而另一个可能要最便宜的路由。这将导致运营商在设置最短路径成本时使用不同的度量（例如，毫秒计的延迟与货币计的成本）。这些网络上的权重没有可比性，因此互联网上的最短路径将得不到明确的定义。

更糟糕的是，一个运营商甚至可能不希望另一个运营商了解它网络的路径细节，因为

权重和路径多少反映出一些敏感信息(比如货币成本), 这些信息代表了一种商业竞争优势。

最后, 互联网可能比构成它的任何一个网络都大。因此, 它或许需要采用层次结构的扩展性较好的路由算法, 即使没有一个网络需要使用层次结构路由。

所有这些因素导致了两级路由算法。在每个网络中, 使用一个域内 (intradomain) 或者内部网关协议 (interior gateway protocol) 进行路由 (“网关” 是 “路由器” 的旧称)。这可能是我们已经描述过的一种链路状态协议。为了让数据包跨越构成互联网的网络, 就需要用到域间 (interdomain) 或外部网关协议 (exterior gateway protocol)。网络可能全部使用不同的域内协议, 但它们必须使用相同的域间协议。在 Internet 上, 域间路由协议称为边界网关协议 (BGP, Border Gateway Protocol)。我们将在下节描述它。

还有一个更重要的术语要介绍。由于每个网络独立于所有其他网络运营, 因此这样的网络通常称为一个自治系统 (AS, Autonomous System)。AS 的良好默认模型是 ISP 网络。事实上, 一个 ISP 网络或许由多个 AS 组成, 如果它管理或收购了多个网络。但两者之间的差异通常不显著。

这两级路由通常没有严格的层次。如果一个庞大的国际网络和一个较小的区域网络都被抽象成一个单一网络, 可能导致极不优化的路径。然而, 暴露出来的有关网络内部路由信息相对太少, 以至于无法找到互联网络上的路由。但这有助于解决所有的复杂性, 改善网络尺度, 并允许运营商使用自选的路由协议来自由地选择网络内部的路由, 而且不需要比较不同网络的权重或者将敏感信息暴露在网络以外。

然而, 到目前为止我们很少提到有关如何在组成互联网的这些网络之间路由。在 Internet 上, 一大决定因素是 ISP 之间的商业安排。每个 ISP 可能因替其他 ISP 承载流量而收取或接收相应的费用。另一个因素在于, 如果国际互联网络的路由需要跨越国界, 各国的法律可能会突然开始发挥作用, 比如瑞典严格的隐私法律条款禁止出口有关瑞典公民的个人资料。所有这些非技术因素都包裹在一个路由策略 (routing policy) 概念中, 控制着自治网络自主选择所用的路由。我们将在描述 BGP 协议时返回到路由策略这个主题上。

5.5.5 数据包分段

每个网络或链路都会限制其数据包的最大长度。这种限制来自多方面的原因, 其中包括:

- (1) 硬件 (比如以太网帧的长度限制)。
- (2) 操作系统 (比如所有的缓冲区都是 512 字节)。
- (3) 协议 (比如, 数据包长度字段中的位数)。
- (4) 遵从某个国家 (或国际) 标准。
- (5) 期望将错误引入的重传次数减少到某种程度。
- (6) 期望防止数据包占用信道时间太长。

所有这些因素导致的结果是网络设计者们无法自由地选择任何他们所期望的最大数据包长度。一些常用技术的最大有效载荷长度: 以太网为 1500 字节, 802.11 协议为 2272 字节, IP 协议更通用一些, 允许数据包的长度最多可达 65 515 字节。

主机一般倾向于传输大的数据包, 因为这样可以降低开销, 比如浪费在头字节上的带宽。当一个大数据包要穿过一个最大数据包尺寸太小的网络时, 一个明显的网络互联问题

就出现了。这个滋扰一直是网络互联的老大难问题，随着从 Internet 上获得的丰富经验，相应的解决方案得到了很大发展。

一种解决方案是首先确保不会发生这个问题。然而，这说起来容易做起来难。源端通常不知道数据包通过网络到达接收方的路径，因此它当然不知道到达接收方的数据包尺寸有多大。这个数据包尺寸称为路径最大传输单元 (MTU, Path Maximum Transmission Unit)。即使源端知道路径 MTU，数据包在无连接网络中也是独立路由的，比如 Internet。这种路由意味着路径可能会突然改变，因而意外地更改路径 MTU。

另一种解决办法是允许路由器将数据包拆分成段，将每个段作为一个独立的网络层数据包发送。不过，正如每个小孩父母都知道，将一个物体转换成小的碎片远比相反的过程更容易（物理学家甚至给这个效果起了个名称：热力学第二定律）。数据包交换网络把段重新整合起来也同样麻烦多多。

将分段重新组成原始的数据包，可以采用两种对立的策略。第一种策略，由“小数据包”网络引起的分段过程对于沿途后续的网络都是透明的，也就是说，从该网络一直到最终的目标途中的每个网络都感觉不到曾经发生过分段，如图 5-42 (a) 所示。在这种方法中，当一个超大数据包到达 G_1 ，该路由器将它分割成多个段；每个段都发向同样的出口路由器 G_2 ，在这里这些段被重新组合起来。按照这种方法，任何一个数据包通过这样的小数据包网络都是透明的。后续的网络根本感觉不到曾经发生过分段。

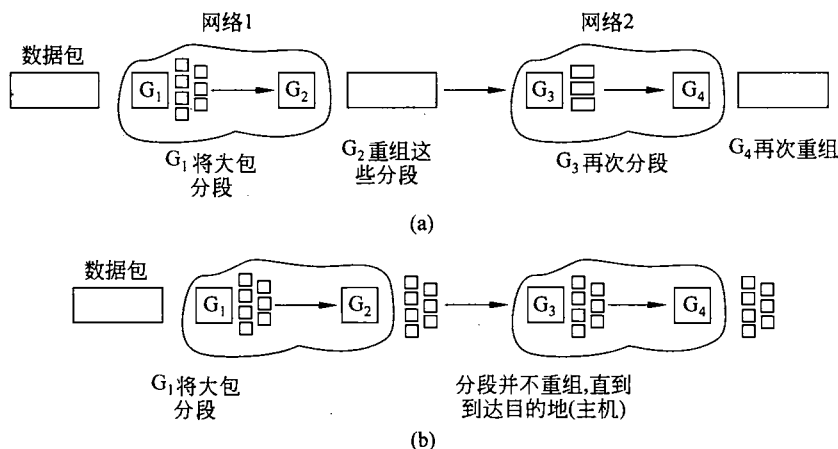


图 5-42
(a) 透明分段；(b) 非透明分段

透明的分段过程非常直接、简单，但是也有一些问题。首先，出口路由器必须知道什么时候它已经接收到了全部的段，所以每个分段中必须提供一个计数字段或者一个“数据包结束”标志位。其次，由于所有的数据包必须经过同一个出口路由器才能进行重组，因此路由受到了限制。由于不允许有些段沿着一条路径到达最终目标，而另一些段沿着一条不相交路径到达最终目标，所以，可能会损失一些性能。更为重要的是，路由器可能不得不做大量的工作。如果不是所有的段都已到达，它还需要缓冲到达的段，并且决定何时丢弃这些段。某些工作可能纯粹是一种浪费，因为当一个数据包需要通过一系列的小数据包网络时，需要多次被分段和重组。

另一种分段策略是避免在任何一个中间路由器上重新组合分段。一旦一个数据包已经

被分段, 则每个段都被当作原始的数据包一样来对待。路由器传递这些段的情形如图 5-42 (b) 所示, 重组过程只在目标主机上进行。

非透明分段的主要优点是路由器所做的工作比较少。IP 就是以这种方式工作的。一个完整的设计要求分段可以重新构建原有数据流的方式编号。IP 采用的设计思想是: 给每个段一个数据包序号 (所有的数据包都携带)、一个数据包内的绝对字节偏移量和一个指明是否到达数据包末尾的标志位。图 5-43 给出了一个例子。虽然这种设计简单, 但有一些吸引人的特性。段到达目的地后可以被放置在一个缓冲区中以便重组, 即使这些分段到达的秩序凌乱不堪; 当段要穿过一个 MTU 更小的网络时, 还可以被路由器再次进行分段, 如图 5-43 (c) 显示的那样; 数据包的重传 (如果所有的段都没有收到) 可以被分割成不同的段。最后, 段可以任意大小, 最小的段是一个字节加上数据包头。在所有情况下, 接收方只需简单地使用数据包的序号和段偏移量即可把数据放置在合适的位置, 并利用数据包结束标志位来确定何时有了一个完整的数据包。

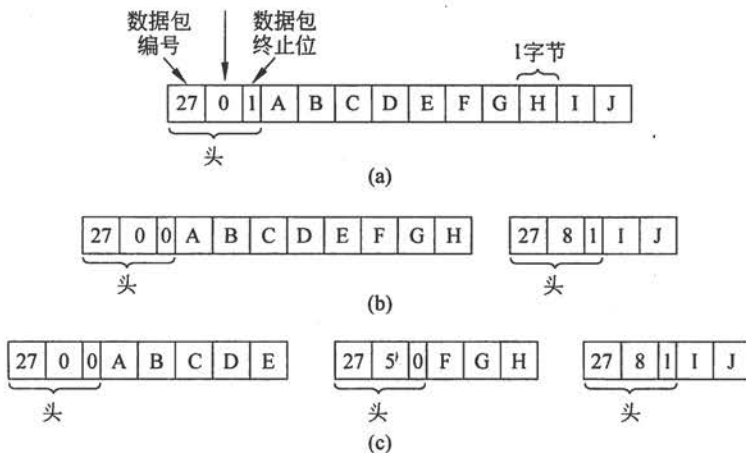


图 5-43 基本数据大小是 1 个字节时的分段

(a) 原始数据包, 包含 10 个字节的数据; (b) 经过一个最大包尺寸为 8 的网络后的分段加上头;
(c) 经过一个大小为 5 的网关后的分段

不幸的是, 这样的设计还是有问题。因为现在某些链路上运载的段的头或许是不必要的。但真正的问题首先还是因为段的存在, 因此开销可能比透明分段高。(Kent 和 Motul, 1987) 认为分段不利于性能, 因为除了增加头开销, 数据包的丢失概率也增加了; 任何一个段的丢失都将导致整个数据包的丢失; 而且对主机而言, 分比不分带来了更大的突发。

这又导致我们回到了最初的解决方案, 就是在网络中避免分段操作, 这种策略被现代 Internet 所采用。这个过程称为路径 MTU 发现 (path MTU discovery) (Mogul 和 Deering, 1990)。它的工作原理如下所示。每个 IP 数据包发出时在它的头设置一个比特, 指示不允许对该数据包实施分段操作。如果一个路由器接收的数据包太大, 它就生成一个报错数据包并发送给源端, 然后丢弃该数据包, 如图 5-44 所示。当源端收到报错数据包, 它就使用报错数据包携带的信息重新将出错数据包分段, 每个段足够小到报错路由器能处理。如果沿着路径前进又遇到一个 MTU 更小的路由器, 那么重复上述过程。

路径 MTU 发现的优点是源端现在知道应该发送多长的数据包了。如果路由和路径 MTU 发生改变, 新的报错数据包将被触发返回, 因而源端将适应新的路径。然而, 仍然需

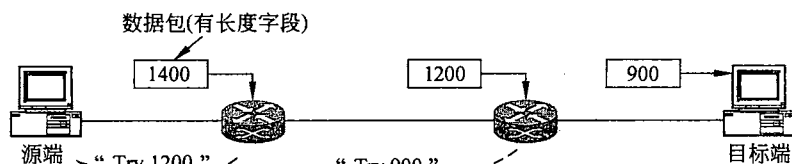


图 5-44 路径 MTU 的发现过程

要在源端和接收方之间实施分段，除非更高层次的协议了解路径 MTU，并把适量的数据传给 IP。TCP 和 IP 通常是一起实现的（正如 TCP/IP 称呼），因而能够传递这类信息。即使其他协议不是这样实现，仍然可以把分段移出网络，让主机承担。

路径 MTU 发现方法的缺点是有可能增加发送数据包的启动延迟。在任何数据被传递到目的地之前，探讨路径并且找出该路径 MTU 所花的时间可能需要不止一个往返延迟。这就引出了一个问题，是否有更好的设计？答案很可能“是的”。考虑这样的设计：每个路由器把那些超出其 MTU 的数据包简单地拦腰截断。这样一来将确保接收方尽可能快地了解 MTU（从传递过来的数据量），同时接收了部分数据。

5.6 Internet 的网络层

现在是时候详细讨论 Internet 的网络层了。但在进入 Internet 细节之前，值得先看看当初驱动其设计并导致其今天成功的原则。现在，很多时候人们似乎已经忘记了这些原则。RFC 1958 列举了这些原则，并对它们进行了讨论，该文档很是值得一读（这是所有协议设计者的必修课——最后应该有个期末考试）。这份 RFC 文档着重描述了由（Clark, 1988）和（Saltzer 等, 1984）提出的想法。现在，我们总结我们所认为的 10 大原则（从最重要的到最不重要的）。

（1）**保证工作**。直到多个原型系统成功地与对方相互通信，方可完成设计或者确定标准。设计者常常首先写出一个 1000 页的标准，并获得批准，过后才发现存在严重的缺陷，根本无法工作。然后他们再编写一个 1.1 版本的标准。这不是正确的工作方式。

（2）**保持简单**。有疑问时应该使用最简单的解决方案。William of Occam 在 14 世纪就提出了这条原则（称为奥卡姆的剃刀）。换成现代术语就是：决斗特性。如果一项特性并非绝对不要，那么就放弃该特性。尤其是，通过组合其他的特性也能够获得同样效果的时候。

（3）**明确选择**。如果有几种方法可以完成同样的事情，则选择其中一种方法。用两种或者多种方法来做同样的事情简直是自找麻烦。通常标准会有多个选项、多种模式或多个参数，因为多个实力强大的参与方坚持认为他们的方法是最好的。设计者应该坚决抵制这种倾向，学会说“不”。

（4）**模块开发**。这条原则直接导致了协议栈的思想，每一层的协议完全独立于所有其他的协议。按照这种方法，如果实际环境中要求改变一个模块或者一层，则其他模块或层都不会受到影响。

（5）**期望异构性**。在任何一个大型的网络中，可能存在不同类型的硬件、传输设施和

应用程序。为了处理它们，网络的设计必须简单、通用和灵活。

(6) **避免静态选项和参数。**如果不可避免要使用参数的话（比如最大数据包长度），那么，最好的办法是让发送方和接收方协商一个值，而不是定义固定的参数值。

(7) **寻找好的而不是完美的设计。**通常设计者有一个好的设计，但是它不能够处理一些怪异的特殊情况。设计者不应该乱改设计，而是坚持这个好的设计，将围绕着特殊情况而展开的工作负担转移到那些强烈需求的人身上。

(8) **严格发送，宽容接收。**换句话说，只发送那些严格符合标准的数据包，但是，容许接收那些不完全符合标准的数据包，并且试图对它们进行处理。

(9) **考虑可扩展性。**如果系统需要有效地处理上百万台主机和几十亿用户，那么，没有一种中心数据库是可以容忍的，同时必须将负载尽可能均匀地分布到所有可用的资源上。

(10) **考虑性能和成本。**如果一个网络的性能很差，或者成本很高，那么没有人会使用它。

现在，让我们离开通用设计原则，开始 Internet 网络的探索之旅。在网络层，可以把 Internet 看作是一种相互关联的网络或自治域（自治系统）集合。没有真正的结构，但存在几个主要骨干网。这些都是由高带宽线路和快速路由器组成。这些骨干网中最大的一个称为**一级网络（Tier 1 networks）**，每个骨干网都与它连接，进而到达其他骨干网。连接到骨干网上的是 **Internet 服务提供商（ISP，Internet Service Provider）**，它为家庭和企业、数据中心和服务器托管设施，以及区域（中级）网络提供 Internet 接入服务。数据中心提供了许多通过 Internet 发送的内容。连接到区域网络的是更多的 ISP、许多大学和公司的局域网和其他边缘网络。图 5-45 给出了一个准分层组织的轮廓图。

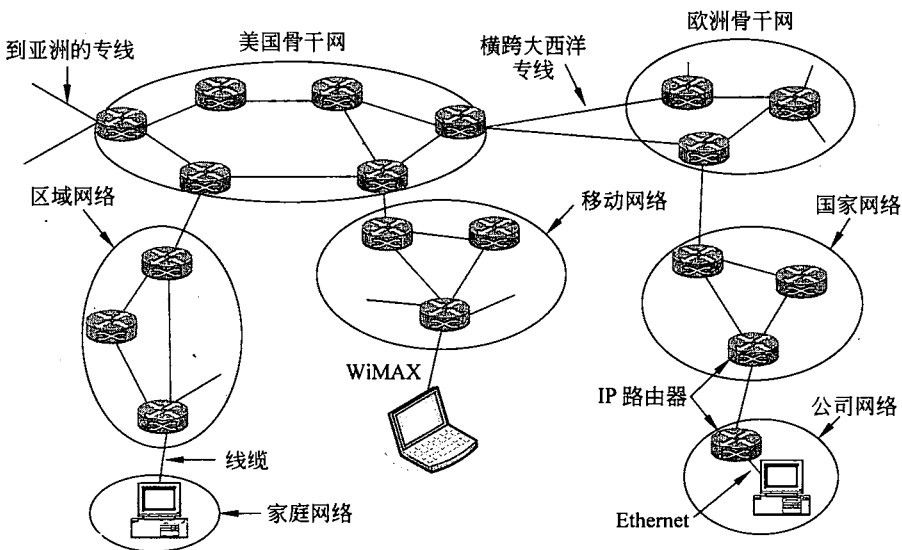


图 5-45 Internet 是互相连接的许多网络

将整个 Internet 黏合在一起的正是网络层协议，即 **Internet 协议（IP，Internet Protocol）**。与大多数老式网络层协议不同的是，IP 协议在设计之初就把网络互联作为目标。应该这样看待网络层：IP 的任务是提供一种尽力而为（best-effort）地把数据包从源端传输到接收方的方法（即不提供任何保证），无须考虑这些机器是否在同一个网络，也不必关心它们之间

是否还有其他网络。

Internet 中的通信过程是这样的。传输层获取数据流，并且将数据流拆分成段，以便作为 IP 数据包发送。理论上，每个数据包最多可容纳 64 KB，但实际上，数据包通常不超过 1500 个字节（因而它们正好可被放到一个以太网帧中）。IP 路由器转发每个数据包穿过 Internet，沿着一条路径把数据包从一个路由器转发到下一个路由器，直到数据包到达目的地。在接收方，网络层将数据交给传输层，再由传输层交给接收进程。当所有的数据段最终都抵达目标机器，它们被网络层重新组装还原成最初的数据报；然后该数据报被网络层传给传输层。

在图 5-45 所示的例子中，家庭网络上的一个主机要穿越四个网络 and 大量 IP 路由器，才能到达位于公司网络上的目标主机。这种情况在实际中并不罕见，而且还有很多更长的路径。在 Internet 上存在着很多冗余连接，骨干网和 ISP 在多个位置相互连接。这意味着两个主机之间存在着许多可能的路径。决定使用哪些路径正是 IP 路由协议的任务。

5.6.1 IPv4 协议

开始学习 Internet 网络层最恰当的方式是从 IP 数据报本身的格式开始。每个 IP 数据报包含两部分，一个头和一个正文，正文部分也称之为有效净荷。头由一个 20 字节的定长部分和一个可选的变长部分组成。图 5-46 显示了 IP 数据报的头格式。IP 数据报头的传输从左到右并从上到下，Version 字段的高序字节最先被传送出去（这就是“big-endian”网络字节序。在 little-endian 字节序机器上，比如 Intel x86 计算机，在传输和接收时需要进行字节顺序的软件转换）。现在回想起来，little-endian 字节序是更好的选择，但在设计 IP 协议时没有人能预测到今天 little-endian 流行计算世界。

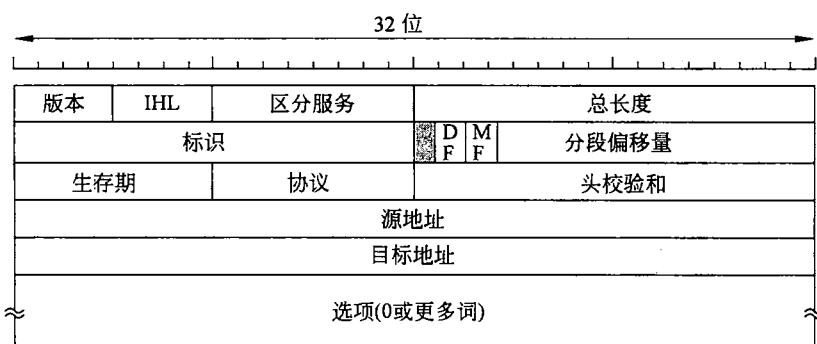


图 5-46 IPv4 (Internet 协议) 头

版本 (Version) 字段记录了数据报属于协议哪个版本。版本 4 主宰着今天的 Internet，这也就是为什么我们从这里开始学习的真正原因。在每个数据报开始包含版本信息，使得版本之间的迁移过程可以持续很长一段时间。事实上，IPv6，即 IP 协议的下一个版本已经被定义了十多年，但它的部署却还刚刚开始不久。当某个国家 2^{31} 人口中每个人都拥有一个台式 PC、一台笔记本电脑或者一个 IP 电话时，最终将不得不使用 IPv6。我们将在本节后面介绍 IPv6。关于版本编号，这里顺便提一下，IPv5 是一个试验性的实时流协议，它一直没有被广泛应用。

由于头的长度不固定，所以头的 IHL 字段指明了头到底有多长（以 32 位字长度为单位）。IHL 的最小值为 5，这表明头没有可选项。该 4 位字段的最大值为 15，把头的长度限制为最大 60 字节，因此选项（Options）字段最多为 40 字节。对于某些选项，比如记录一个数据包路径的选项，40 字节往往太小，这使得这样的选项其实没有什么用处。

区分服务（Differentiated services）字段是少数几个在意义上随岁月（轻微）改变的字段之一。该字段最初称为服务类型（Type of service）。它曾经并且仍然用来区分不同的服务种类。可靠性和速度的各种组合都是可能的选择。对于数字化的话音数据，加速传递优先于精确传递；对于文件传输，正确传输比加速传输更加重要。最初时，服务类型字段包含 6 位，其中 3 位表示优先级，3 位代表主机最关心的是延迟、吞吐量或可靠性中的哪一个。然而，没人真正知道路由器用这些位做什么，因此这些位空着许多年没人用。在设计区分服务时，IETF 承认自己的失败，并重新启用这个字段。现在，前 6 位用来标记数据包的服务类别，我们在本章前面描述过的加速服务和确保服务；后 2 位用来携带显式拥塞通知信息，比如数据包是否经历了拥塞，我们在本章的拥塞控制部分描述了显式拥塞通知。

总长度（Total length）字段包含了该数据报中的所有内容，即头和数据。最大长度是 65 535 个字节。目前情况下，这样的上界还是可以容忍的，但在未来网络中，可能需要更大的数据报。

标识（Identification）字段的用途是让目标主机确定一个新到达的分段属于哪一个数据报。同一个数据报的所有段包含同样的标识值。

接下来是一个未使用的位，这很令人惊讶，因为 IP 头中可供使用的地方实在是很珍贵。作为愚人节开的一个玩笑，（Bellovin, 2003）提出用该位来检测恶意流量。因为已知带有该“邪恶”位的数据包是由攻击者发送，因此路由器可把它丢弃，这样可以大大简化安全控制。不幸的是，网络安全并不是这么简单。

接下来的两个 1 位字段与分段有关。DF 代表“不分段”（Don't Fragment）标志位。这是针对路由器的一条命令，它不允许路由器分割该数据报。最初，该字段用来支持没有能力组装还原数据包的主机。现在该字段可用在发现路径 MTU 过程中，路经 MTU 是能经过路经而无须分段的最大数据包。通过在发出的数据包中设置 DF 位，发送方知道这个数据包要么完整地到达目的地，要么有个报错消息反馈回来。

MF 代表“更多的段”（More Fragments）标志位。除了最后一个段以外，其他所有的段都必须设置这一位。它的用途是接收方可以知道什么时候一个数据报的所有分段都已经到达了。

分段偏移量（Fragment offset）字段指明了该段在当前数据报中的位置。除了数据报的最后一个段以外，其他所有段的长度必须是 8 字节的倍数。由于该字段有 13 位，所以每个数据报最多有 8192 个段，由此支持 Total length 字段限制的最大数据报。Identification、MF 和 Fragment offset 这 3 个字段协同工作，可用来实现 5.5.5 节描述的分段操作。

生存期（Time to live）字段是一个用于限制数据包生存期的计数器。这里的计数单位最初设置为秒，因此最大的生存期为 255 秒。在每一跳上该计数器必须被递减，而且，当数据报在一台路由器上排队时间较长时，该计数器必须多倍递减。实际上，它只是跳计数器，当它递减到 0 时，数据包就被丢弃，并且路由器给数据包的源主机发回一个报警包。此项特性可以避免数据包永远逗留在网络中，有时候当路由表被破坏之后可能会发生这样

的事情。

当网络层组装完成一个完整的数据包之后，它需要知道该如何对它进行处理。协议（Protocol）字段指明了该将它交给哪个传输进程。TCP 是一种可能，但是 UDP 或者其他协议也有可能。协议的编号在整个 Internet 是全球统一的。RFC 1700 中列出了以前的协议和其他分配的编号，现在的协议编号包含在一个位于 www.iana.org 的在线数据库中。

由于头携带了诸如地址那样致命的信息，因此它用自己的校验和加以保护，即头校验和（Header checksum）字段。校验算法的执行过程是这样的：当数据到达时，所有的 16 位（半字）累加起来，然后再取结果的补码。该算法的目的是到达数据包的头校验和计算结果应该为 0。这样的校验和对于检测数据包穿过网络时是否发生错误非常有用。请注意，在每一跳必须重新计算头校验和字段，因为至少有个字段总是不断在改变（即生存期字段），但是，采用一些技巧可以加速计算。

源地址（Source address）字段和目标地址（Destination address）字段表示源网络接口和目标网络接口的 IP 地址。我们将在下一小节介绍 Internet 地址。

选项（Options）字段的设计意图是提供一种途径，允许后续版本协议包含一些原设计中没有出现的信息，以便实验人员尝试新的想法、避免为那些不常使用的信息分配头字段。选项具有可变长度。每个选项的第一个字节是一个标识码，它标明了该选项类别。有的选项后面跟着一个 1 字节的选项长度字段，然后是一个或多个数据字节。选项字段用来将整个选项长度填充到 4 字节的倍数。最初设计时定义了 5 个选项，如图 5-47 所示。

选项	描述
安全性	标明数据报的安全级别
严格源路由	给出数据报遵循的完整路径
松散源路由	给出一些不能错过的路由器
记录路由	要求每个路由器加上自己的 IP 地址
时间戳	要求每个路由器加上自己的 IP 地址和时间戳

图 5-47 某些 IP 选项

安全（Security）选项指明了信息的秘密程度。理论上，军用路由器可能使用这个字段来指定路由时，不允许通过某些在军事上被认为是“敌对”的国家。实际上，所有的路由器都忽略该选项，所以，它仅有的实际用途是帮助间谍们更加容易找到好的材料。

严格源路由（Strict source routing）选项给出了从源到目标的完整路径，其形式是一系列 IP 地址。数据报必须严格地遵循这条路径向前传输。对于系统管理员，这个选项非常有用，他们可以在路由表被破坏时发送紧急数据包，或者用这个选项来测量时间。

松散源路由（Loose source routing）选项要求该数据包穿越所指定的路由器列表，并且要求按照列表中的顺序前进；但是，在途中也允许经过其他路由器。通常情况下，该选项往往只提供少数几个路由器，用来强迫数据包走一条特殊的路径。例如，为了强迫一个从伦敦到悉尼的数据包必须向西走而不是向东走，该选项可以指定纽约、洛杉矶和檀香山的路由器作为路由必经节点。当出于政治或者经济的考虑而要求经过或者避开某些国家的时候，这个选项最有用。

记录路由（Record route）选项告诉沿途的路由器，将自己的 IP 地址附加到可送字段中。

这样系统管理员就可以跟踪路由算法中的错误（比如，“为什么从休斯敦到达拉斯的数据包要经过东京？”）。当 ARPANET 刚开始建立时，没有一个数据包会经过 9 个以上的路由器，所以 40 字节的选项足够了。正如前面所提到的，现在 40 字节显得太小了。

最后，时间戳（Timestamp）选项类似于记录路由选项，只不过每个路由器除了记录自己的 32 位 IP 地址以外，还要记录一个 32 位时间戳。对于网络测量，这个选项也是最有用的。

如今，IP 选项已失宠。许多路由器忽略它们或者不能有效地处理它们，它们作为一种罕见案例被搁置了起来。也就是说，它们只是得到部分支持，但很少被使用。

5.6.2 IP 地址

IPv4 的一个明确特征是它的 32 位地址。Internet 上的每台主机和每个路由器都有一个 IP 地址，可用在 IP 数据包的 Source address 和 Destination address 字段。重要的是要注意，一个 IP 地址并不真正指向一台主机，而是指向一个网络接口，所以如果一台主机在两个网络上，它必须有两个 IP 地址。然而，大多数主机都连在一个网络，因而只有一个 IP 地址。与此相反，路由器有多个接口，从而有多个 IP 地址。

前缀

与以太网地址不同的是 IP 地址具有层次性。每个 32 位地址由高位的可变长网络和低位的主机两部分数据组成。同一网络上（比如以太局域网）的所有主机，其地址的网络值是相同的。这意味着一个网络对应一块连续的 IP 地址空间，这块地址空间就称为地址的前缀（prefix）。

IP 地址的书写方式是点分十进制表示法。按此格式，4 个字节中的每个写成十进制，取值范围从 0 到 255。例如，32 位十六进制地址 80D00297 写成 128.208.2.151。前缀的书写给出了块的最低 IP 地址和块的大小。块大小由网络部分的位数决定；地址中主机部分的剩余位数可以有所变化。这意味着，块的大小必须是 2 的幂。按照惯例，网络地址的书写格式是前缀 IP 地址后跟一个斜线，斜线后是网络部分的位长度。在我们的例子中，如果前缀包含 2^8 个地址，所以留下了 24 位用于网络部分，写成 128.208.0.0/24。

因为前缀长度仅从 IP 地址无法推断出来，路由协议必须把前缀携带给路由器。有时候，前缀很简单地由长度描述，比如“/16”，读音为“slash 16”。前缀长度相当于网络部分中 1 的二进制掩码。以这种格式书写时称为子网掩码（subnet mask），它可以与一个 IP 地址进行 AND 操作，以便提取出该 IP 地址的网络部分。在我们的例子中，子网掩码为 255.255.255.0。图 5-48 显示了一个前缀和一个子网掩码。

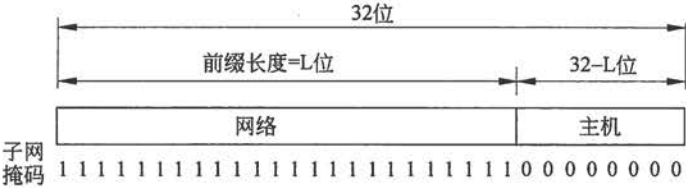


图 5-48 IP 的前缀和子网掩码

层次化的地址具有显著的优点和缺点。前缀的主要优势是路由器仅仅根据地址的网络部分即可转发数据包，只要每个网络都有一个唯一的地址块。对路由器来说网络地址的主机部分并不重要，因为同一网络上的所有主机都在同一个方向，只有当数据包到达它们的目的地网络后才被转发到正确的主机。这样可使得路由表远远小于按每个 IP 地址索引所需要的大小，考虑到 Internet 的主机数量已接近十亿，这对于每个路由器需要维护的路由表来说实在是太大了。然而，通过分层的方式路由器只需要保持约 30 万条前缀的路由。

虽然使用层次路由使得 Internet 路由规模化，但它有两个缺点。首先，一个主机的 IP 地址取决于它位于网络上的位置。以太网地址可用于世界上的任何地方，但每个 IP 地址属于一个特定的网络，路由器只能传递注定要到该网络上某个地址的数据包。比如移动 IP 这样的设计必须支持主机在网络之间移动，但同时保持相同的 IP 地址。

第二个缺点在于层次结构浪费了地址，除非精心管理地址空间。如果给网络分配（太）大块的地址，将有（很多）被分配掉但不会使用。这种分配将没有多大意义，如果有大量的地址空间，这倒没什么关系。然而，二十年以前人们就认识到 Internet 的巨大增长正迅速消耗着自由地址空间。IPv6 就是地址短缺问题的解决方案，但直到它被广泛部署，否则分配 IP 地址都将面临巨大的压力，所以必须有效地使用地址。

子网

为了避免冲突，网络地址的管理由一个称为 **Internet 域名和地址分配机构**（ICANN，Internet Corporation for Assigned Names and Number）的非营利性公司负责。ICANN 一次把部分地址空间授权给各区域机构，这些机构再把 IP 地址发放给 ISP 和其他公司。这就是一家公司获得一块 IP 地址的过程。

然而，这个过程仅仅是故事的开始，因为 IP 地址的分配随着企业的成长也在不断地消耗着。我们曾经说过，按前缀路由需要网络中的所有主机具有相同的网络号。随着网络的发展这个属性可能会产生问题。例如，我们用一所大学开始举例说明有关地址问题。/16 分配给计算机科学系，供它以太网上的所有计算机使用；一年后，电机工程学系想上 Internet，不久艺术系紧跟其后。这些系应该用什么 IP 地址？另外申请一个地址块不仅远离学校，而且可能既昂贵又不方便。此外，早已经分配获得的 /16 足够 60 000 台主机的地址。这可能是为了将来有发展空间才申请的，但至少现在还没到那个规模，为同一所大学再分配另一块地址显然是一种浪费。因此，需要一种不同的地址组织。

问题的解决方案是，在内部将一个网络块分成几个部分供多个内部网络使用，但对外部世界仍然像单个网络一样。这就是所谓的**子网划分**（subnetting），分割一个大型网络得到的一系列结果网络（比如以太网）称为**子网**（subnet）。正如我们在第 1 章中提到的那样，你应该意识到这个词的新用法和旧的用法有冲突，子网的以前含义是指网络中的所有路由器和通信线路的集合。

图 5-49 显示了子网如何帮助解决我们例子中的问题。一个 /16 地址空间被分割成几片。这种分割并不要求均匀，但每片必须对齐以便可以把较低的任何位用作地址的主机部分。在这种情况下，块的一半（一个 /17）分配给了计算机科学系，四分之一分配给了电机工程学系（一个 /18），八分之一（一个 /19）分配给了艺术系，剩余的八分之一未分配。了解地址块如何分割的不同方式是看以二进制表示的结果前缀：

计算机科学系:	10000000	11010000	1 xxxxxxx xxxxxxxx
电子工程学系:	10000000	11010000	00 xxxxxx xxxxxxxx
艺术系:	10000000	11010000	011 xxxxx xxxxxxxx

这里的竖线 (|) 表示子网号和主机部分的边界。

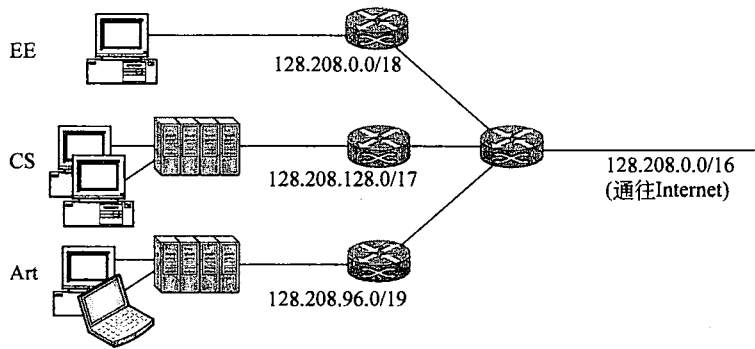


图 5-49 将 IP 前缀进一步细分为网络和子网

当一个 IP 数据包到达主路由器，路由器如何知道应该将它转发到哪个子网？这必须进入到深入了解前缀的细节。一种方法是每个路由器有一张具有 65 536 个项的表，该表告诉它校园网络上每台主机对应的输出线路是哪条。但是，这样会破坏掉使用层次路由带来的可扩展性。相反，路由器只需简单了解校园网络的子网掩码。

当数据包到达时，路由器会查看该数据包的目标地址，并检查它属于哪个子网。具体做法是：路由器把数据包的目标地址与每个子网的掩码进行 AND 操作，看结果是否对应于某个前缀。例如，考虑一个发往 IP 地址 128.208.2.151 的数据包。为了看它是否属于计算机科学系，我们把该目标地址与 255.255.128.0 进行 AND 操作，获得前 17 位（即 128.208.0.0），并且检查它们是否匹配计算机系的前缀地址（即 128.208.128.0）。显然它们不匹配。然后再检查前 18 位，在与电机工程学系的子网掩码 AND 操作后，我们得到 128.208.0.0，这恰好匹配电机工程学系的前缀地址，所以数据包被转发到通往电机工程系网络的接口。

如果有必要，子网的划分还可以改变，只需要更新校园网内部路由器上的所有子网掩码即可。在网络外面，子网的划分是不可见的，因此分配一个新的子网不需要联系 ICANN 或者改变任何外部数据库。

CIDR——无类域间路由

即使 IP 地址按块分配，因此地址空间得以有效地使用，但还有一个问题依然存在：路由表爆炸。

一个组织（比如大学）中位于网络边缘的路由器必须为每个自己的子网设立一个表项，该表项告诉路由器使用哪条线路到达该子网。对于通往组织外部某个目的地的路由，这些路由器可以利用简单的默认路由把数据包发送到通往 ISP 的线路上，ISP 把组织与 Internet 其余部分相连。其他目的地址必定在外面的某个地方。

ISP 和骨干路由器之间的路由没有这样奢侈，这些骨干路由器位于 Internet 中间。它们必须知道通过哪些方式可到达每个网络，这里没有简单的默认路由可用。这些核心路由器处在一个 Internet 默认自由区（default-free zone）。没有人真正知道有多少网络连接到

Internet, 但它肯定是一个很庞大的数字, 可能至少 100 万。这样的规模会产生一个巨大的路由表。用计算机标准来衡量这个数字并不大, 但我们要意识到路由器转发每一个数据包时都必须查询这张表, 而大型 ISP 的一个路由器可能每秒要转发数百万个数据包。必须用专门的硬件和快速内存才能以如此高的速率处理数据包, 一般的通用计算机是无法胜任此项工作的。

此外, 路由算法要求每个路由器与其他路由器交换有关它能到达的地址信息。表越大, 需要通信和处理的信息量也越大, 并且处理时间至少随表的大小呈线性增长。更多的通信增加了丢失部分路由表的可能性, 至少有暂时丢失的可能, 这又可能导致路由的不稳定。

路由表的问题本来可通过一个更深的层次来解决, 类似电话网那样的多层结构。例如, 让每个 IP 地址包含一个国家、州/省、市、网络和主机这些字段可能会奏效。然后, 每个路由器只需要知道如何去每个国家、本国的州或省、本州或本省的城市以及本城的网络。不幸的是, 这种解决方案需要比 32 位多得多的 IP 地址, 而且地址的使用缺乏效率 (并且, 列支敦士登也必须像美国一样, 使用许多位的地址)。

幸运的是, 我们可以做一些事情来减小路由表的长度。我们可以运用与子网划分相同的观点: 不同地点的路由器可以知道一个给定 IP 地址的不同大小前缀。然而, 不是将一块地址分割成子网, 相反, 在这里我们把多个小前缀的地址块合并成一个大前缀的地址块。这个合并过程称为路由聚合 (route aggregation), 由此产生的较大前缀地址块有时称为超网 (supernet), 以便有别于地址块的分割。

有了地址聚合, IP 地址可包含大小不等的前缀。同样一个 IP 地址, 一台路由器把它当作/22 的一部分对待 (该地址块包含 2^{10} 个地址), 而另一台路由器把它当作一个更大的/20 一部分对待 (其中包含 2^{12} 个地址)。这是因为每个路由器有相应的前缀信息。这个设计和子网划分协同工作, 统称为无类域间路由 (CIDR, Classless Inter-Domain Routing), 发音为 “cider”。它的最新版本由 RFC 4632 说明 (Fuller 和 Li, 2006 年)。这个名字突出了与有类别的地址层次编码的不同, 稍后我们将简要介绍这点。

为了使 CIDR 更加易于理解, 让我们来考虑一个例子, 有一块地址从 194.24.0.0 开始, 可用的 IP 地址有 8192 个。假设剑桥大学需要 2048 个地址, 它分配获得的地址范围从 194.24.0.0 到 194.24.7.255, 掩码为 255.255.248.0, 这是一个/21 前缀。接下来, 牛津大学申请 4096 个地址。由于 4096 个地址的块必须位于 4096 的字节边界上, 所以, 牛津大学申请的地址不可能从 194.24.8.0 开始。相反, 它们获得了从 194.24.16.0 至 194.24.31.255 的地址块, 子网掩码为 255.255.240.0。最后, 爱丁堡大学申请 1024 个地址, 它获得了从 194.24.8.0 至 194.24.11.255 的地址块, 掩码为 255.255.252.0。图 5-50 概括了这些地址块的分配情况。

大学	第一个地址	最后一个地址	多少地址	前缀
剑桥	194.24.0.0	194.24.7.255	2048	194.24.0.0/21
爱丁堡	194.24.8.0	194.24.11.255	1024	194.24.8.0/22
(可用)	194.24.12.0	194.24.15.255	1024	194.24.12.0/22
牛津	194.24.16.0	194.24.31.255	4096	194.24.16.0/20

图 5-50 一组分配的 IP 地址

在默认自由区的所有路由器现在都被告知这 3 个网络的 IP 地址。靠近大学的路由器针

对每个前缀可能需要发送到不同的出境线路，所以它们需要自己的路由表中为每个前缀设立相应的表项。作为例子，图 5-51 给出了伦敦路由器示意图。

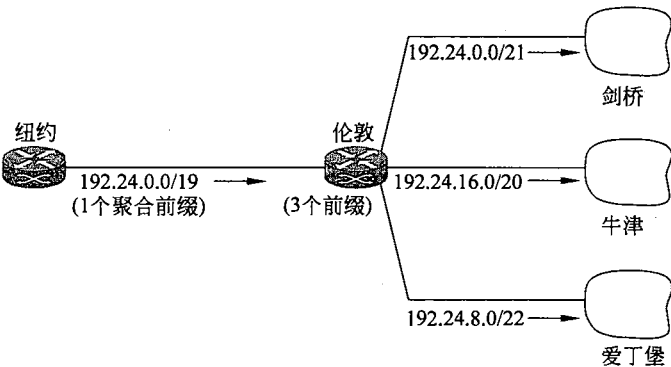


图 5-51 IP 前缀的聚合

现在让我们从位于纽约的一个远程路由器的角度来看这 3 所大学。3 个前缀的所有 IP 地址都应该从纽约（或更一般地，从美国）发送到伦敦。伦敦的路由进程注意到这个，并且将 3 个前缀合成一个聚合表项，即前缀 194.24.0.0/19，然后传递给纽约路由器。这个前缀包含了 8K 个地址，并涵盖了 3 所大学的地址和其他未分配的 1024 个地址。通过聚合，3 个前缀已经减少为一个，由此减少了必须告知纽约路由器的前缀，并且纽约路由器的路由表表项。

当聚合功能被启用后，这个过程完全自动。这依赖于给 Internet 分配什么样的前缀，而不是给网络分配地址的管理员行动。聚合技术在 Internet 中被大量使用，已经把路由器表的大小减少到目前的约 200 000 万前缀。

更为扭曲的是前缀允许重叠。规则是数据包按最具体路由的方向发送，即具有最少 IP 地址的最长匹配前缀（longest matching prefix）。最长匹配前缀路由提供了有益的灵活性，正如从图 5-52 显示的纽约路由器的行为。该路由器仍使用单一聚合前缀把三所大学的流量发送到伦敦。然而，该前缀中的先前那块未用地址现在已经分配给了旧金山的网络。一种可能是纽约路由器保持四个前缀，其中三个前缀的数据包发送到伦敦，第四个前缀的数据包发送到旧金山。相反，最长匹配前缀路由可以用图中显示的两个前缀来转发。一个总的前缀指示把整个地址块的流量发到伦敦；一个更具体的前缀用来指示该大前缀的一部分流量发往旧金山。有了最长匹配前缀规则，到旧金山网络的 IP 地址的流量将被发送到通往旧金山的出境线路，并且发往大前缀中所有其他 IP 地址的流量将被送往伦敦。



图 5-52 纽约路由器中的最长匹配前缀路由

从概念上讲，CIDR 的工作原理如下所述。当一个数据包到达时，路由器扫描路由表以便确定目的地是否在前缀的地址块内。有可能多个具有不同前缀的表项得到匹配，在这

种情况下,使用具有最长前缀的表项。因此,如果有一个匹配/20掩码的表项,同时还有一个匹配/24掩码的表项,则使用/24表项来查询数据包的出境线路。然而,如果真正一个表项一个表项地扫描表,这个过程将非常冗长乏味。相反,人们设计了复杂的算法来加快地址匹配过程(Ruiz-Sanchez等,2001)。商用路由器使用了定制的VLSI,这些算法被嵌入到了硬件中。

分类和特殊寻址

为了帮助你更好地理解为什么CIDR如此有用,我们简要介绍之前的地址设计方案。在1993年以前,IP地址被分为图5-53列出的5个类别。这种分配已经称为分类寻址(classful addressing)。

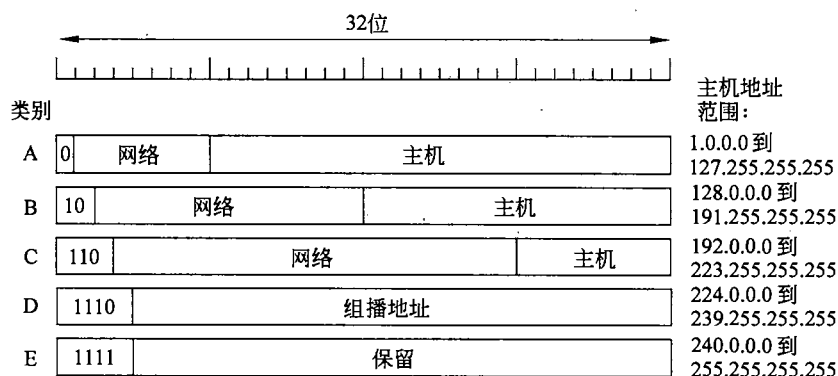


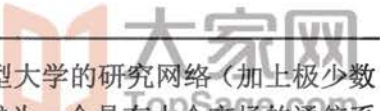
图 5-53 IP 地址格式

A、B、C类地址格式分别允许多达128个网络,每个网络1600万台主机;16384个网络,每个网络65536台主机;以及200万个网络(比如LAN),每个网络多达256台主机(不过有些地址是特殊的)。IP地址分类同时支持组播(D类地址),即数据包被直接发送给多台主机。以1111开头的地址是保留地址,以备将来使用。在IPv4地址空间已经耗尽的情况下,使用它们将是很有价值的。不幸的是,许多主机将不再把它们视为有效地址而接受,因为它们已经被禁锢了这么久,很难教会旧主机一些新花样。

这是一个层次化的设计,但与CIDR不同于,这里的地址块大小是固定的。超过20亿个地址,通过类别划分来组织地址空间会浪费数百万的地址。尤其是,真正的罪魁祸首是B类网络。对于大多数组织,具有1600万个地址的A类网络太大,而具有256个地址的C类网络又太小。一个具有65536个地址的B类网络恰到好处。在Internet民间传说中,这种情况称为3只熊问题(犹如《Goldilocks and the Three Bears》(Southey, 1848))。

在现实中,一个B类地址对大多数组织而言还是过于庞大。研究表明,超过半数的B类网络少于50台主机。一个C类网络足够应付得过来。但毫无疑问,每个组织都会申请一个B类地址,它们想到总有一天自己的网络规模将超过8位主机地址空间的范围。现在回想起来,给C类网络分配10位的主机号而不是8位主机号或许更好,因为这样的分配使得每个网络可以拥有1022台主机。如果真是这样,大多数组织可能用一个C类网络就能解决问题,而且将有差不多50万个C类网络(相对于B类网络只有16384个)。

很难指责Internet的设计者没有提供更多(和更小)的B类地址。在当时决定创建这



三个类别的地址时，Internet 还只是一个连接美国主要研究型大学的研究网络（加上极少数公司和从事网络研究的军事基地）。没有人认为 Internet 将成为一个具有大众市场的通信系统，而且还能和电话网络相媲美。当时，人们毫无疑问地说：“美国有大约 2000 所大专院校。即使它们都连接到 Internet，甚至其他国家的多所大学也加入进来，我们也永远达不到 16 000，因为整个世界没那么多的大学。而且，主机号是一个整数字节可加快数据包的处理速度”（当时完全由软件完成）。也许有一天人们回顾过去，会责备设计电话号码的人并说：“谁这么白痴。为什么不把地球编号包括在电话号码中？”但在当时，似乎没有什么必要。

为了处理这些问题，引入了子网这个概念，以便在一个组织内部灵活分配地址块。后来，又引入了 CIDR 技术来减小全局路由表的大小。今天，表明一个 IP 地址是否属于 A、B 或 C 类网络的标志位已不再使用，但在文献中对这些类的引用仍然很普遍。

为了看清楚地址分类如何使得转发更为复杂，先考虑在旧的分类系统中它有多简单。当一个数据包到达路由器，IP 地址的一个副本被右移 28 位，产生一个 4 位的类别号；然后 16 路分支把数据包归纳到 A、B、C（及 D 和 E）类，A 类有八种情况，B 类有四种情况，以及 C 类有两种情况；然后每个类别用掩码取出 8 位、16 位或 24 位的网络号，并且右对齐成 32 位的字。然后用该网络号查询 A、B 或 C 表，通常查询 A 和 B 类网络采用索引方式，而查询 C 网络则采用哈希算法。一旦找到对应的表项，即可查到相应的出境线路，进而转发数据包。这种查询操作比最长匹配前缀操作简单，因为最长匹配前缀面对的 IP 地址不是固定前缀长度，因而不能使用一个简单的查表操作。

D 类地址可继续用于 Internet 组播。实际上，或许可以更准确地说它们开始被用于组播了，因为过去 Internet 还没有广泛部署过组播。

还有几个其他地址有特殊的含义，如图 5-54 所示。IP 地址 0.0.0.0，这是最低的地址，由主机在启动时使用。这个地址意味着“这个网络”或者“这个主机”。全 0 的 IP 地址作为网络号指的是当前网络。这些地址允许机器在不知道网络号的情况下访问自己所在的网络（但它们必须知道网络掩码包括多少个 0）。由全 1 或 255.255.255.255 组成的地址，是最高的地址，用来标识指定网络中的所有主机。它允许在本地网络上广播，通常是局域网。具有正确的网络号和主机字段全 1 的地址允许机器向在 Internet 任何地方的遥远局域网发送广播数据包。然而，许多网络管理员禁用了此功能，主要原因在于这是一个安全隐患。最后，所有 127.xx.yy.zz 形式的地址保留给回环测试用。发送到该地址的数据包并没有被真正放在线路上，它们如同入境数据包一样在本地处理。这样在发送方不知道网络号的情况下，可以给主机发送数据包，这对测试网络软件很有用。

0 0	本机
0 0 ... 0 0 主机	本地网络的主机
1 1	在本地网络广播
网络 1 1 1 1 ... 1 1 1 1	在远程网络广播
127 任何内容	回环

图 5-54 特殊 IP 地址